

Creating a Simulation Model

Creating a Simulation Model

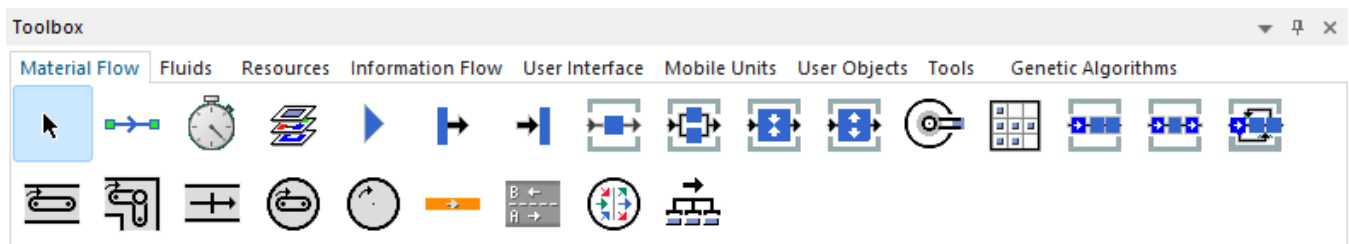
You will create your simulation model by inserting instances of the built-in objects and of objects you yourself or a colleague designed, the so-called user-defined or application objects, from the *Class Library* into the *Frame* named **Model** in the folder **Models**.

By creating user-defined objects you can develop libraries that meet the modeling requirements of your company. The most important built-in objects are: the *Frame*, the active and the passive material flow objects, the mobile objects (MUs), the *Method* object, lists and tables, and the *EventController*.

- The *Frame* is the object in which you create your simulation model. You will insert the objects that represent the stations in your plant, into the *Frame* named **Model**.
- To run the simulation you will use *EventController* that starts, stops and resets the simulation run. You can re-use each and every simulation model you created as an application object in other simulation models.

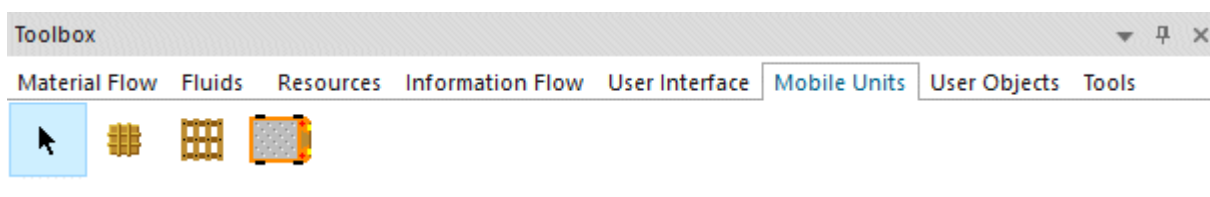
By inserting *Frames* into *Frames* you create a randomly nested **hierarchy** of models. This way you can build and structure simulation models that exactly match the plant you are simulating, and break down complex tasks into manageable chunks.

- You can derive or duplicate each object and re-use it as often as required, which introduces an important time-saving key feature of *Plant Simulation*, namely **inheritance**: A derived object, i.e., a child object, retains a controllable link to the template object, i.e., the parent object. In many cases the immense advantage of inheritance regarding productivity and the low number of errors only becomes clear at a later point in time, when you actively work with your simulation models. Naturally you can also deactivate inheritance for individual features by clicking the inheritance check box, so that it looks like this .
- The **active material flow objects** transport and/or actively process mobile objects (MUs), i.e., parts, within the simulation models.

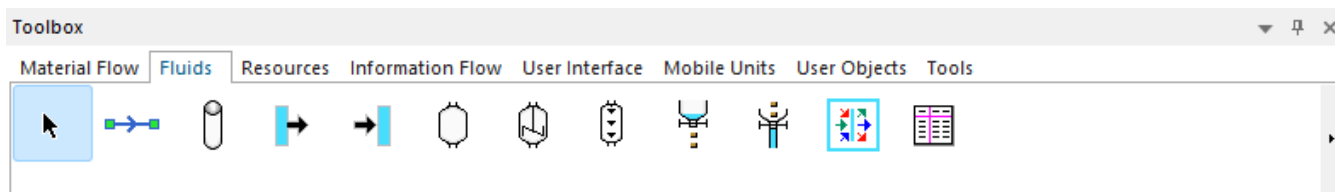


The **passive material flow objects** do not actively process parts. They store parts and represent tracks on which the processed parts are moved.

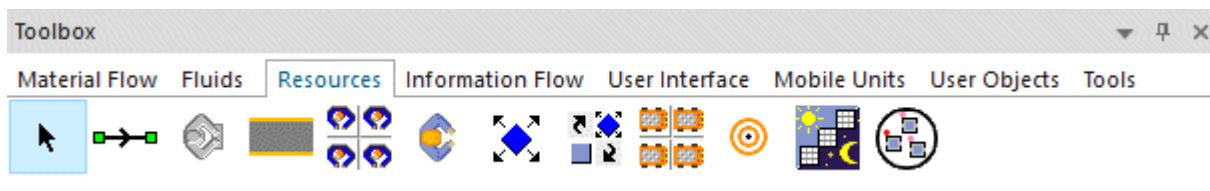
- The **mobile objects (MUs)** represent the parts that are created in, stored in, transported in, processed in and removed from your simulation model.



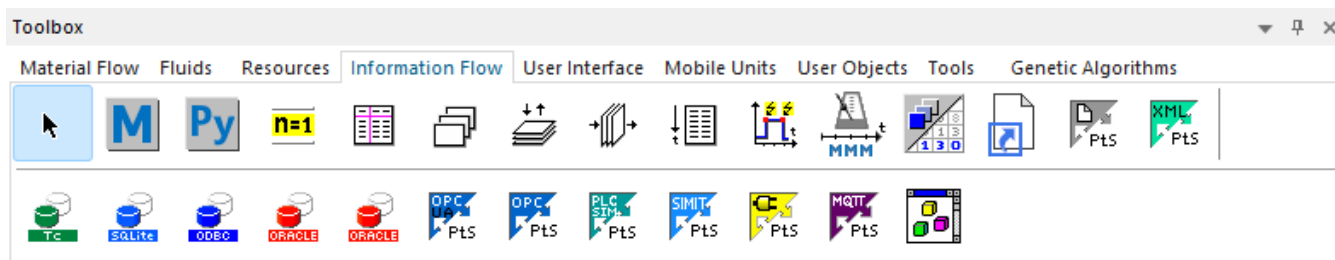
- The **fluid objects** serve for simulating free-flowing materials, be they in liquid, gaseous, or pourable form. The fluid objects are suited for the food and beverages processing, and for the pharmaceutical industries.



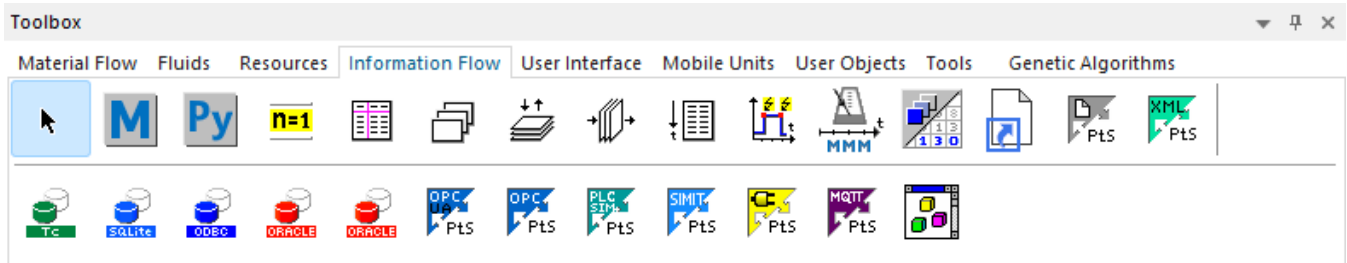
- The **resource objects** are intended for modeling how and when *Workers* move from the *WorkerPool* to the *Workplaces* at the stations.



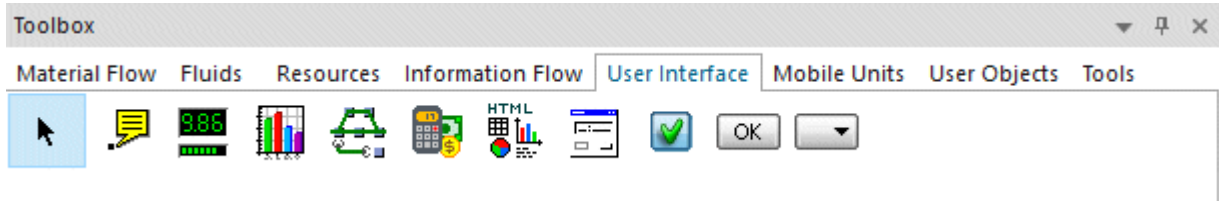
- In objects of type *Method* you can program the actions that you need to take place when the conditions you program become true. For this we integrated the powerful programming language *SimTalk* into *Plant Simulation*. The *Methods* are either activated as controls by the material flow objects or by other *Method* objects. The *Method* also provides the *Method Debugger* to locate errors and allow you to debug and correct faulty code.



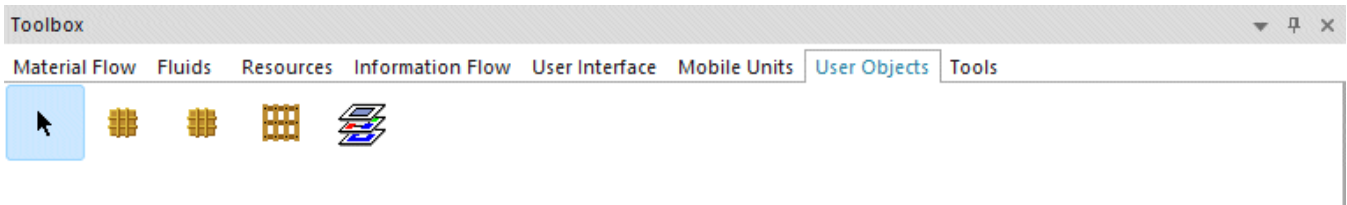
- **Lists and tables** ensure the random exchange of information between all objects. The powerful processing of information in *Plant Simulation* is the result of a variety of functions, data types and operators.



- The objects *Chart* and *Report* allow you to graphically display the statistical values that the material flow objects collect during the simulation runs.



- With the object *Dialog* you can create a dialog box similar to the built-in dialog boxes that *Plant Simulation* provides. This way you can provide a simple user interface for complex simulation models other users work with. You can also prevent another user from manipulating a *Frame*. To accomplish this, insert a *Method* object as an **open control** into the *Frame*. Double-clicking the *Frame* will then not open it any longer, but call the *control method* in which you might, for example, program it to open a dialog, where the user can then select the settings.
- In the folder **UserObjects** you can save and provide objects, which you yourself created and add them to the tab **User Objects**. Here *Plant Simulation* also saves those objects which you derive or duplicate in the *Class Library* if you do not want to work with the built-in class objects.

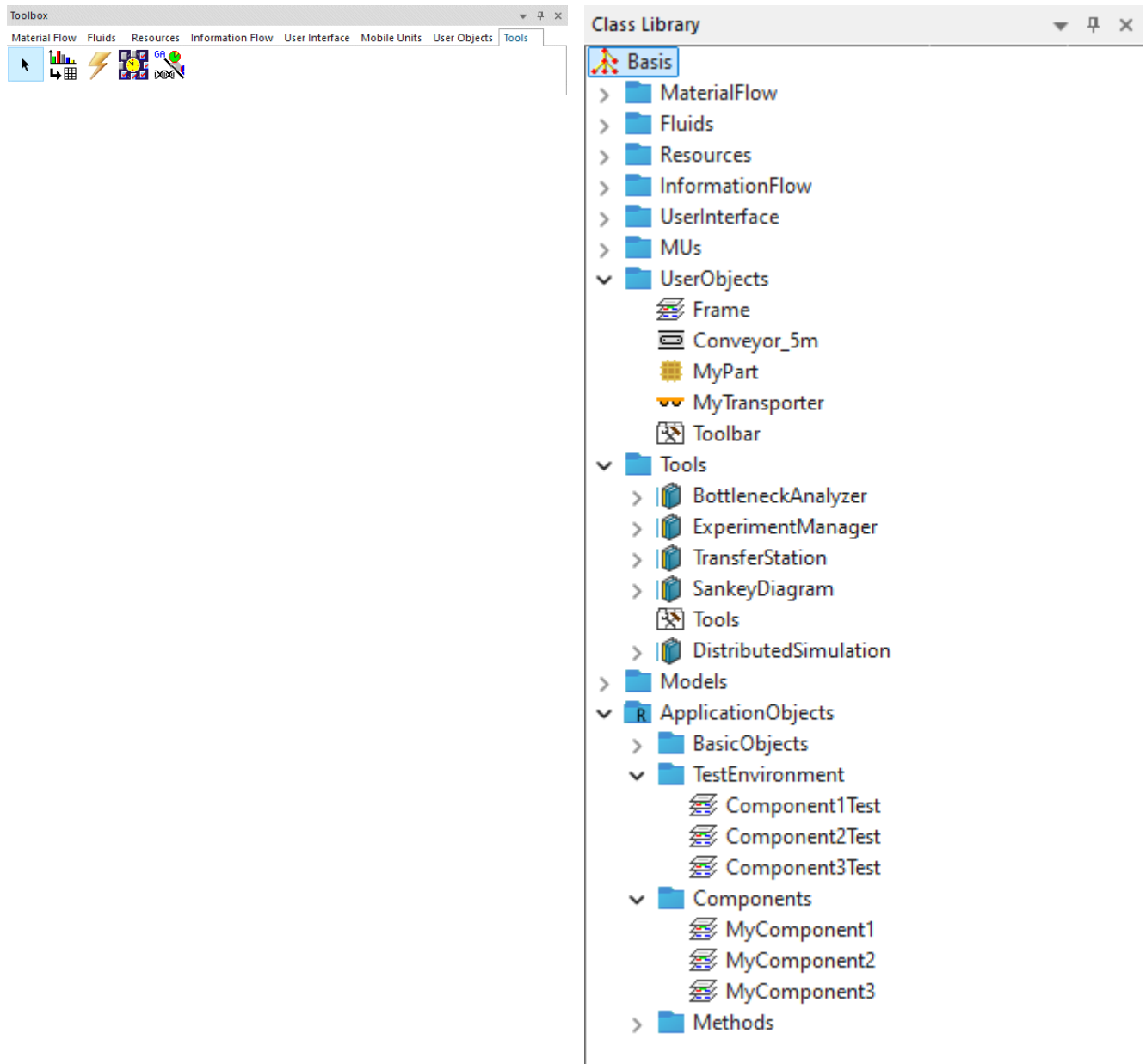


Note

We recommend to not change the default settings of the built-in objects if at all possible. When you customize these objects and when you merge models or update the *Class Library* at a later point in time, *Plant Simulation* discards your changes and restores the default settings. For this reason you have to customize the objects anew.

If you need objects with customized settings, we recommend to derive the respective object/objects in the *Class Library* and then continue working with this class. *Plant Simulation* adds these objects by default to the folder **UserObjects**. You can also add them to the tab **User Objects** in the **Toolbox**.

- With the objects *BottleneckAnalyzer* and *SankeyDiagram* you can analyze and evaluate the results the simulation runs provided. Use the *ExperimentManager* to define how *Plant Simulation* executes the simulation experiments.



- Naturally you can also exchange data with other programs and processes using the interfaces we provide. For integration purposes with other software it is important that data can be exchanged not only at the beginning and the end of the simulation run but also during the simulation run.

When you create your simulation models, you will more than likely combine these two modeling strategies:

- The **top-down strategy** is characterized by a roughly structured model that you break down step-by-step. The more refined the model becomes, the more detailed it becomes. This approach requires a large degree of abstraction. The big picture becomes obvious quickly though.

- The **bottom-up strategy** is characterized by a detailed model that you created by using prefabricated components with a rather small degree of abstraction. This enables for a quick analysis of the details, but makes it harder to recognize the big picture.

Compare [Create a Simulation Model](#)

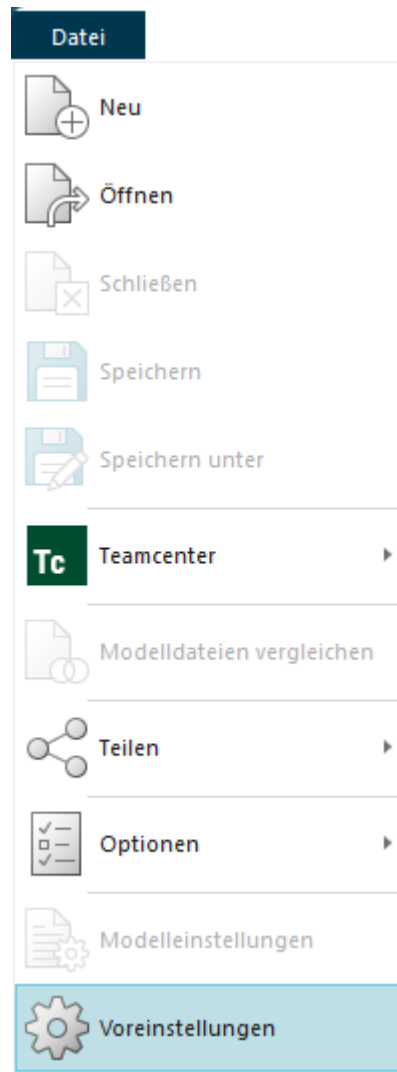
Back to [Modeling the Material Flow](#)

Creating a German Model with English Identifiers

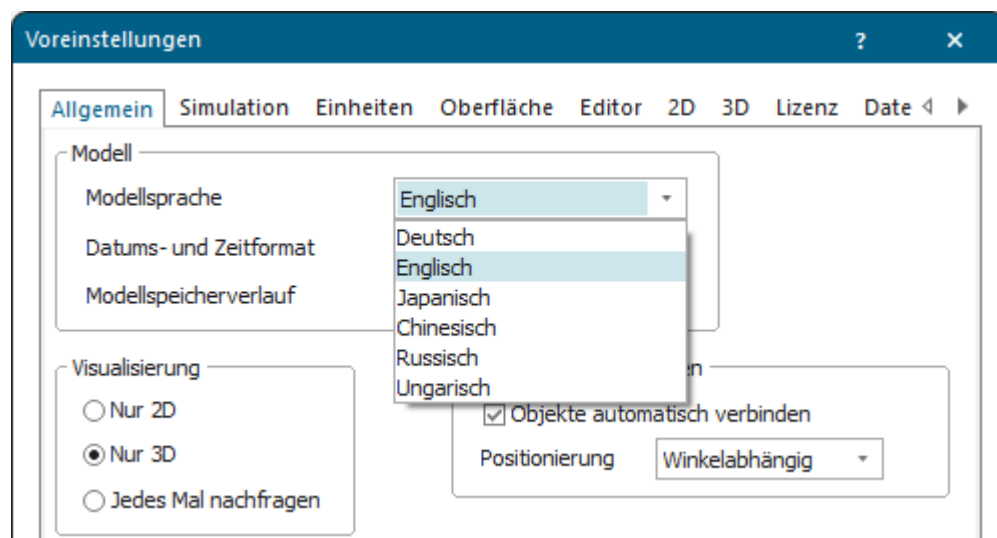
As a rule you will work with the German version of *Plant Simulation* in German-speaking countries. If you are working in an internationally-active company though, one of your requirements might be to create the simulation model with English identifiers so that your colleagues in other countries can expand the model.

As the interpreter, which compiles the instructions of the programming language *SimTalk* at runtime, can process the German and English names of the *SimTalk* commands, nothing is easier than that:

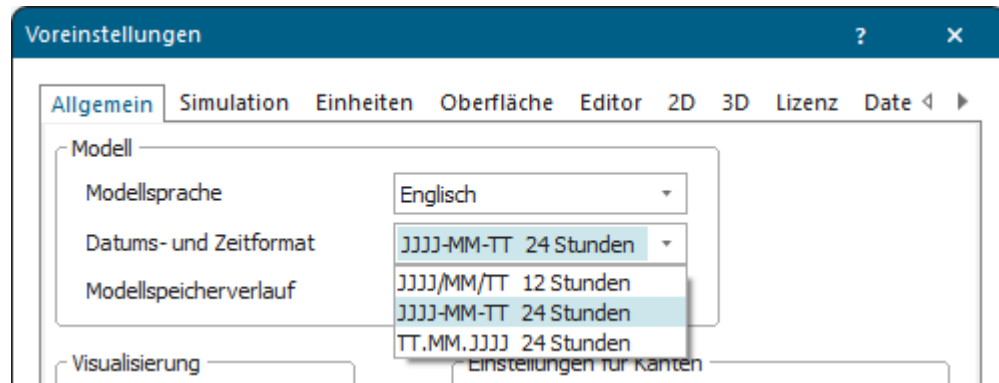
- Start *Plant Simulation*.
- Select **Datei/File** > **Voreinstellungen/Preferences** before opening a model.



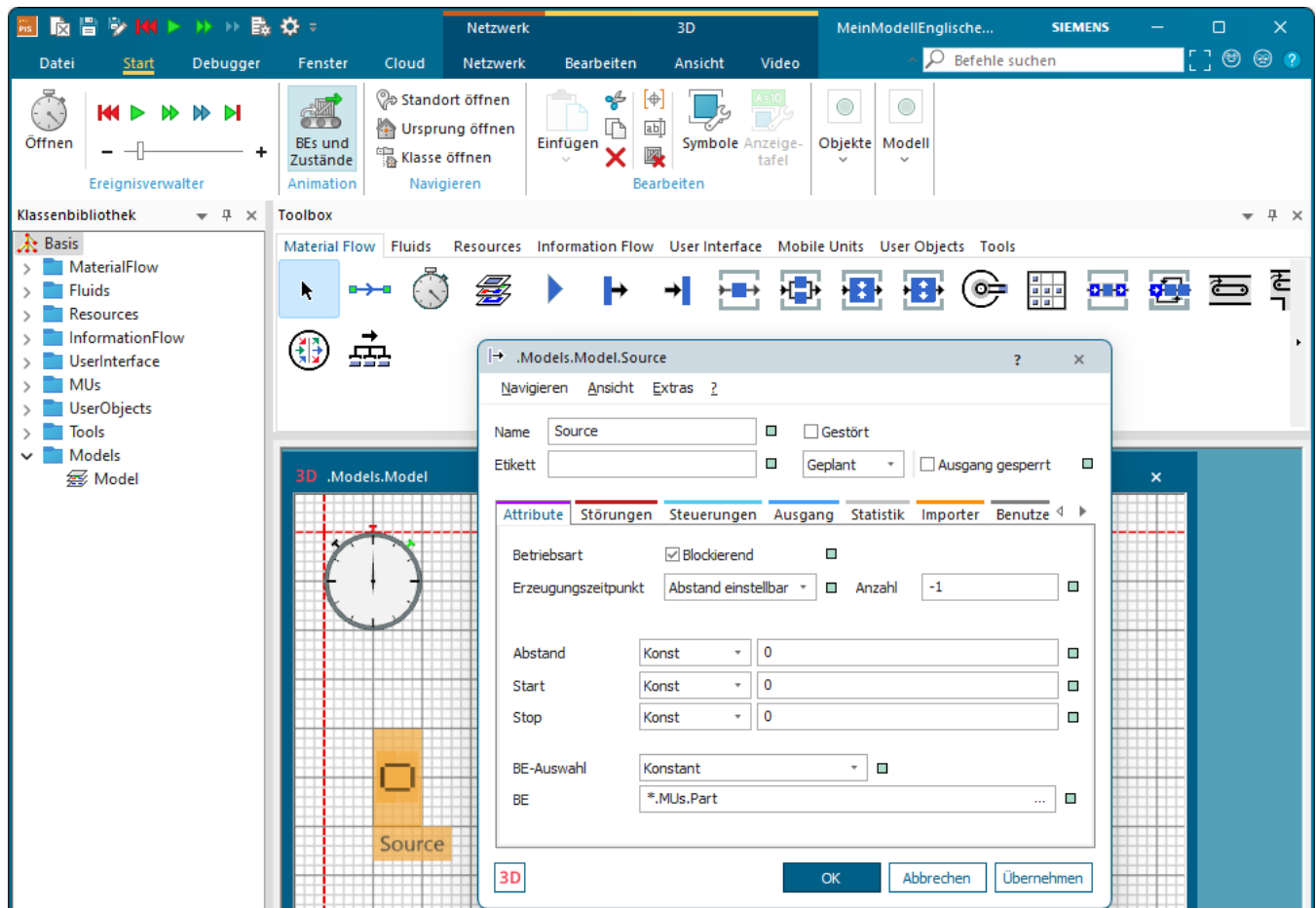
- Click the tab **Allgemein/General**. Select **English** as the language of the model.



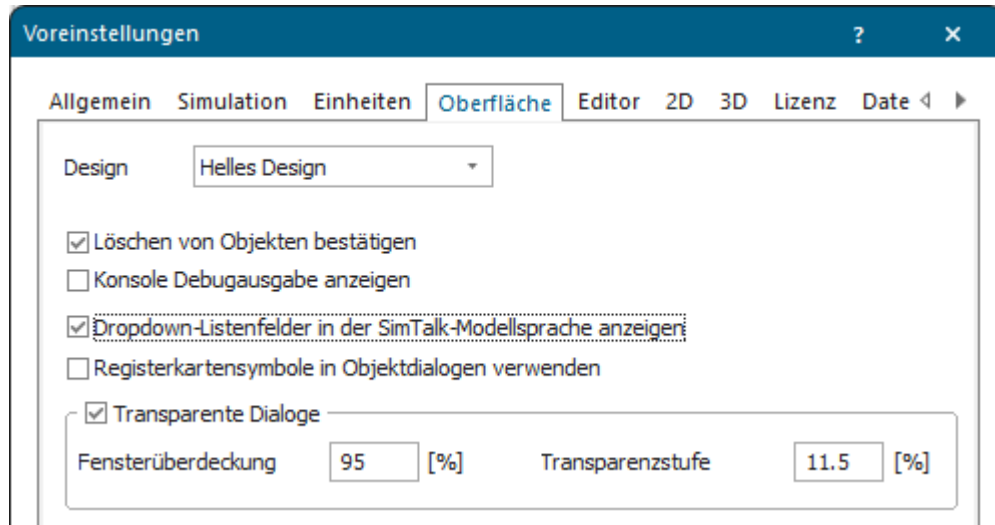
- *Plant Simulation* shows the content of the window **Show Attributes and Methods** in **English**.
- *Plant Simulation* shows the names which the command **Auto Complete** in the *Method-Editor* suggests, in **English**.
- If the plant in the destination country uses the AM/PM time format (12 hours), you can also select this.



- Then click **Create New Model** on the **Start** page.
- *Plant Simulation* shows the names of the folder, which contain the objects, and the names of the objects in **English**, while the menus and the names of the dialog items in the dialogs are still **German**.



Activate **Listenfelder in der SimTalk-Sprache des Modells anzeigen/Display Down-list Boxes in the SimTalk Language of the Model** if you would like to program methods and thus would like to see which values the attributes and methods return. For the model language **German** the expression `Einzelstation.Bearbeitungszeit.Typ` (`Station.ProcTime.Type`) returns **Konst**, for the model language **English** it returns **Const**. Show the list boxes in the user interface language of *Plant Simulation* if you are, for example, working with an **Application License**. Then the contents of the drop-down list boxes is shown in the same language as the other dialog items.



- The German *Plant Simulation Help* shows the **English** names of the methods, attributes, and read-only attributes next to the **German** names. This makes it easier to program methods with **English** identifiers.

fügeBeobachterHinzu [SimTalk] / addObserver

Fügt einen Beobachter zum mit <Pfad> bezeichneten Objekt hinzu.

Typ

Methode

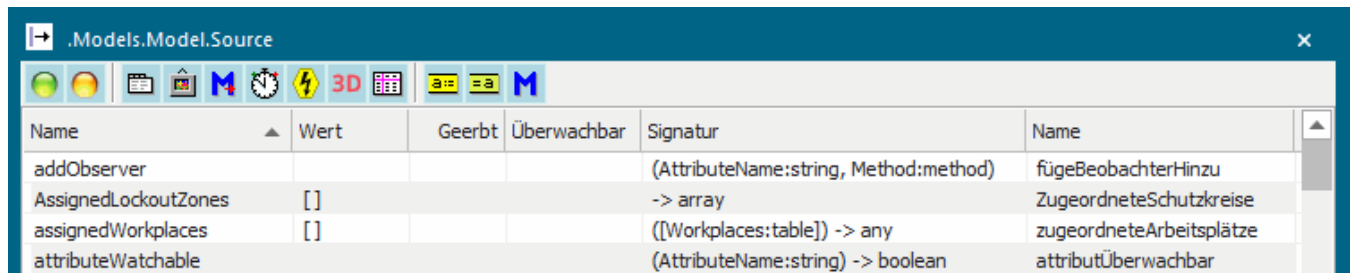
Syntax

```
<Pfad>.fügeBeobachterHinzu(Attributname:string, Methode:object)
```

Kopieren

Parameter

- Der Parameter *Attributname* des Datentyps *String* bezeichnet den Namen des **überwachten Attributs** oder des **überwachten Nur-Lese-Attributs**.
- The window **Show Attributes and Methods (F8)** also shows the **English** names of the methods, attributes, and read-only attributes.



Name	Wert	Geerbt	Überwachbar	Signatur	Name
addObserver				(AttributeName:string, Method:method)	fügeBeobachterHinzu
AssignedLockoutZones	[]			-> array	ZugeordneteSchutzzkreise
assignedWorkplaces	[]			([Workplaces:table]) -> any	zugeordneteArbeitsplätze
attributeWatchable				(AttributeName:string) -> boolean	attributÜberwachbar

[Back to Creating a Simulation Model](#)

Introducing Classes

Introducing Classes

Imagine that, for example, you have to find the optimum type of store for a production plant. First, you have to come up with different types of stores that might work for the specific plant, such as a fully automated high bay warehouse, a manually operated shelving system, etc. To be able to compare the different types of stores, you are going to build several simulation models, execute simulation runs, and suggest the type of store with the best cost/gain relation.

Your simulation models are going to be variants of a single basic model, i.e., you are going to use the same production plant and are going to modify the store.

When creating the model variants you are going to build the basic model first. In conventional systems you would then copy this basic model until you arrive at the number of variants you need. In *Plant Simulation*, on the other hand, you will inherit this basic model, which we call the **parent model**, and get **child models**. The main difference between a copied and an inherited model is that a child model recognizes which parent it is derived from while a copied model knows nothing about its origin.

Now you are going to insert the different types of stores into the model variants you created. If you now find a modeling error while modeling the basic model or if a specification changed, a conventional system requires changing all copies, which is time-consuming and error-prone.

In *Plant Simulation*, on the other hand, you make the change once in the parent model. It then immediately propagates all of your changes to all of its children, provided you did not change the setting in the child model. This saves a considerable amount of time and the hassle of manually updating a number of sub-models.

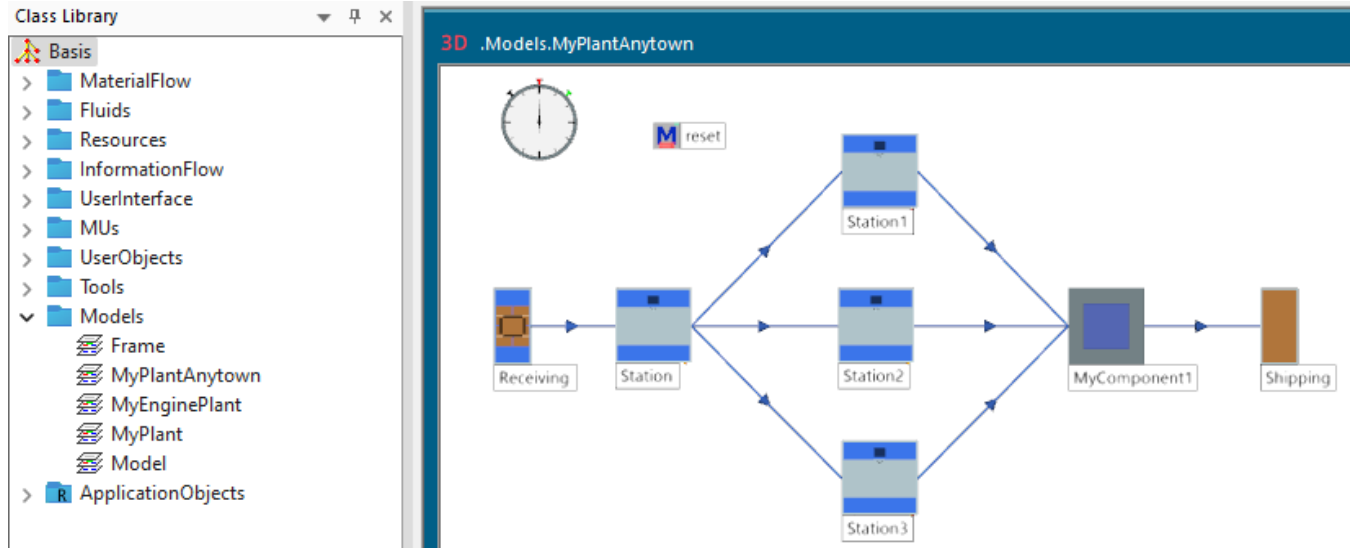
[Back to Creating a Simulation Model](#)

Classes, Subclasses, and Instances

When creating your simulation model, you will work with classes, subclasses, and instances.

When creating your simulation model, keep in mind that:

- A **Class [general description]** is the template for an *instance*, which you insert into a *Frame*. All objects in the **Class Library**, those we built-in as well as those you yourself define, are *class* objects. You can add more detail to these classes and thus create new classes that inherit properties from the master classes.



By changing a property in the class object, you change that property for all objects you derived from this class. This is much less error-prone than having to change the same property for each and every individual object over and over again.

Note

We recommend to not change the default settings of the built-in objects if at all possible. When you customize these objects and when you merge models or update the *Class Library* at a later point in time, *Plant Simulation* discards your changes and restores the default settings. For this reason you have to customize the objects anew.

If you need objects with customized settings, we recommend to derive the respective object/objects in the *Class Library* and then continue working with this class. You can then place these objects into the folder **User Objects** and add them to the tab **User Objects** in the **Toolbox**.

- A *class* passes all of its properties on to an instance you derive from it.
- A *class* passes those of its properties on to a *subclass* you derive from it, for which you do not deactivate inheritance. Remember that a button that looks like this  means that inheritance is active.

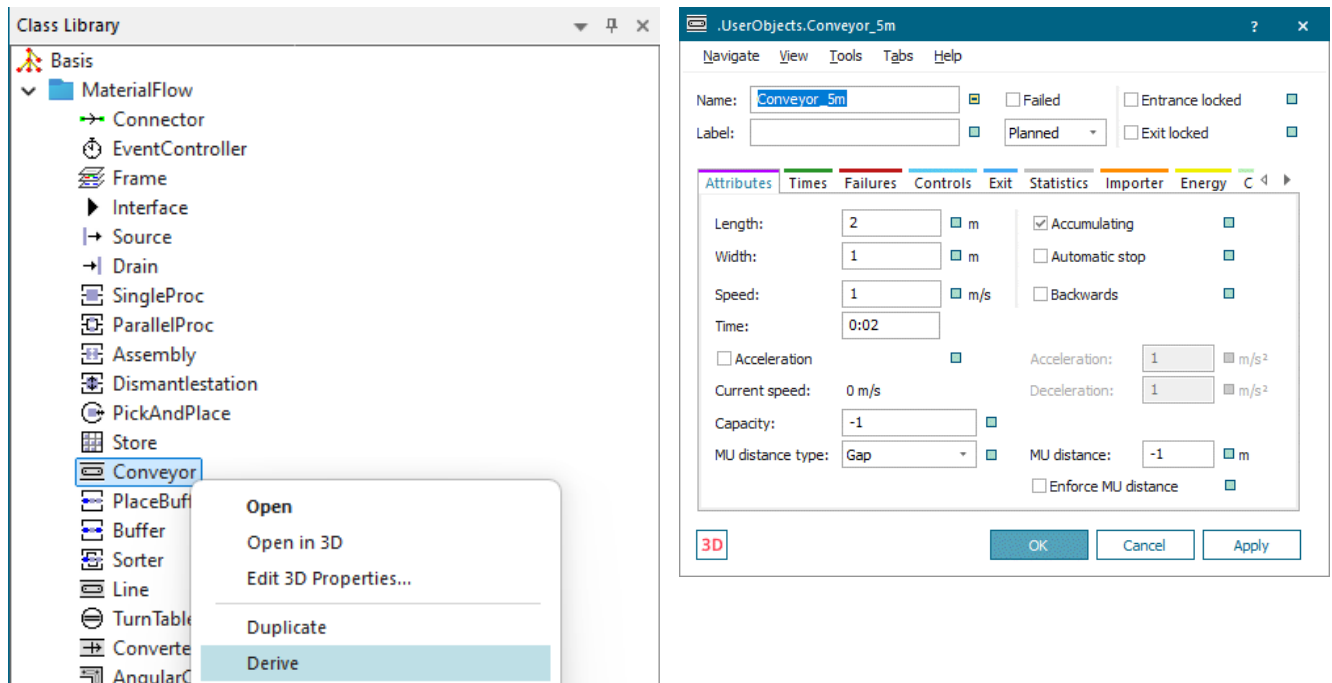
Note

Inheritance only works from *class* to *subclass/instance*, not the other way around!

- A **Subclass [general description]** is an object in the *Class Library*, which inherits some, but not all, of its properties from another *class*. By deactivating inheritance for certain dialog items, you can define properties that only apply to this subclass. An example of subclasses could be several conveyors that only differ in their length.

To **create a subclass from a class**, right-click the class in the *Class Library* and select **Derive**. *Plant Simulation* places the derived object in the folder **UserObjects**. When you open the dialog of the

derived object, you will notice that inheritance is active for all dialog items, denoted by a button that looks like this . Then, change the identifying property of the subclass. In our example below we derived the object *Conveyor* to create the subclass. We renamed it to *Conveyor_5m* and changed its length to 5 meters.



Instead, you can hold down **Ctrl** and **Shift**, and click the object you want to derive an instance from with the left mouse button. Then, hold down the mouse button, drag the object to the location of your choice, for example into another folder in the *Class Library*, and release both keys and the mouse button.

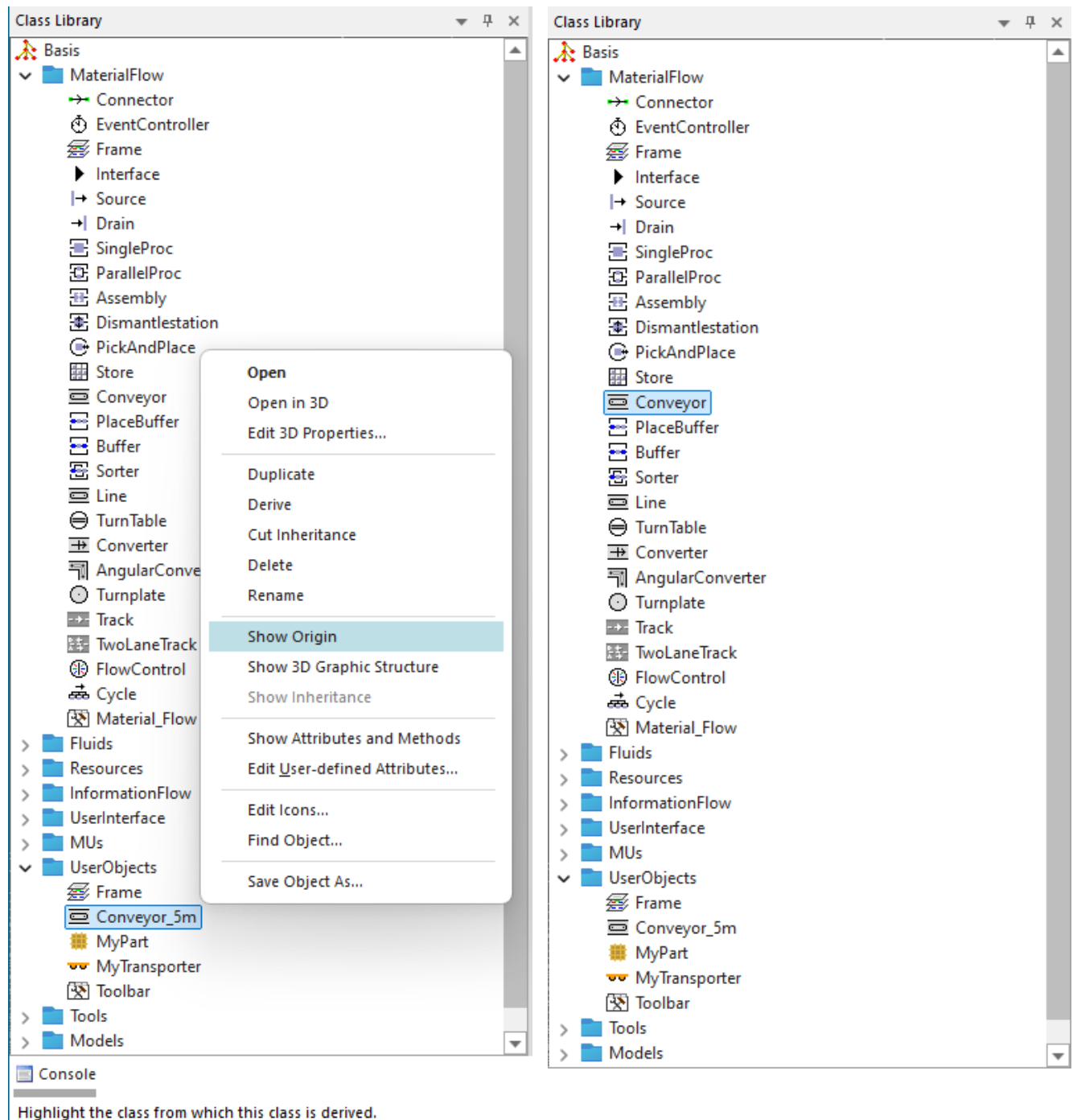
- To **create a copy** from a class, right-click the class in the *Class Library* and select **Duplicate**. When you open the dialog of the duplicated object, you will notice that inheritance is deactivated for all dialog items, denoted by a button that looks like this .

Instead, you can hold down **Ctrl**, and click the object you want to duplicate with the left mouse button. Then, hold down the mouse button, drag the copy to the location of your choice in the *Class Library*, and release **Ctrl** and the mouse button.

- An **Instance [general description]** is an object, which you insert into your simulation model from the *Class Library* or the *Toolbox* by dragging a class object to a *Frame* and dropping it there.

The objects use class relations and origin relations to inherit their properties from other objects:

- The object inherits all of its basic properties from its **class** object. This includes settings it has by default, built-in *Methods*, basic functionalities, etc. The *class* is the object in the *Class Library* you instantiated the selected object from.
- The object inherits its settings from the **origin** object, provided you did not change them locally within the object. The *origin* of the selected object is the object you derived it from.



Making full use of the potentials of inheritance saves a considerable amount of time and effort during modeling.

See also

[Inheritance \[general description\]](#)

Back to [Introducing Classes](#)

Replacing and Merging Objects with Drag-and-Drop

While creating your simulation model , you can replace or merge objects with drag-and-drop.

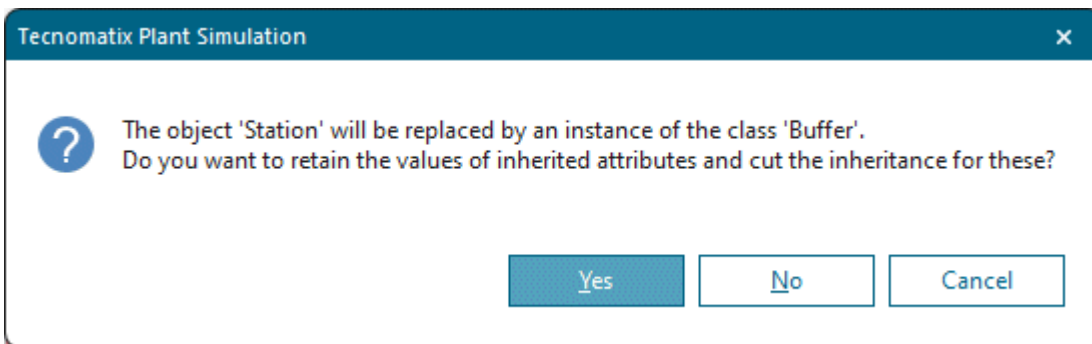
You can:

- **Replace an instance with a class**

Hold down **Alt** while dragging the class object on the instance and dropping it there. This way you can, for example, replace a *Station* with an *AssemblyStation*. *Plant Simulation* preserves all connected *Connectors*, but deletes all *MUs* on the object.

Note

Plant Simulation also lets you copy the attribute values of the previous instance into the new instance. When you select to not do this, the new instance is in the same state as an instance you inserted from the *Class Library*, i.e., all built-in attributes are inherited and all user-defined attributes and sensors, which you defined in the previous instance, will be discarded.



Note

When replacing an instance of an object with the class of another object, *Plant Simulation* retains the existing name. This prevents you from having to change the existing name in a control or a table, for example.

- **Merge classes**

Hold **Alt** and the left mouse button down. Then drag the replacing object in the *Class Library* over the class to be replaced, and drop it there.

Compare [Merge Classes, Example](#)

Compare [Exchange Classes, Example](#)

Back to [Introducing Classes](#)

Using Inheritance

Using Inheritance

Inheritance enables one class or object to incorporate data or behavior of another class or object.

Inheritance has a number of applications, among them:

- **Specialization** of existing classes or objects. Here, the new class or object has data or behavior aspects, which are not part of the inherited class
- **Extension** to provide additional data or behavior features. In contrast to the case of specialization, with extension the new data or behaviors could have been provided in the inherited class because they are generally applicable to all instances of the class.
- **Code re-use** to allow a new class to re-use code, in our case any of the settings you selected, which already exists in another class.

The check box to the right of a dialog item turns inheritance  **Active**  for that item in the dialogs of the *Plant Simulation* objects off or on.

- A button that looks like this  means that inheritance is turned on. Then the object inherits, i.e., uses, the value from the origin object from which you derived it. When you change the value of the origin object, *Plant Simulation* also changes the value of the derived object.

Let us illustrate this with an example. We derive the *Station* in the *Class Library* and insert it from the *Class Library* into the *Frame Model1*. Then, we derive *Model2* from *Model1*. The origin of the *Station* in *Model2* is the *Station* in *Model1*. When we now change the value of an attribute that inherits its value in the *Station* in *Model1*, the *Station* in *Model2* also applies this value.

- A button that looks like this  means that inheritance is turned off. The values you enter only apply to the current object.

The minus sign in the orange button  is intended to assist our vision-impaired users to recognize that inheritance is turned off.

Notes

The object might display the current value only after you click **Apply**.

In the object **Dialog** you can recreate inheritance using the feature **New Check Box**.

Back to [Introducing Classes](#)

Video on YouTube

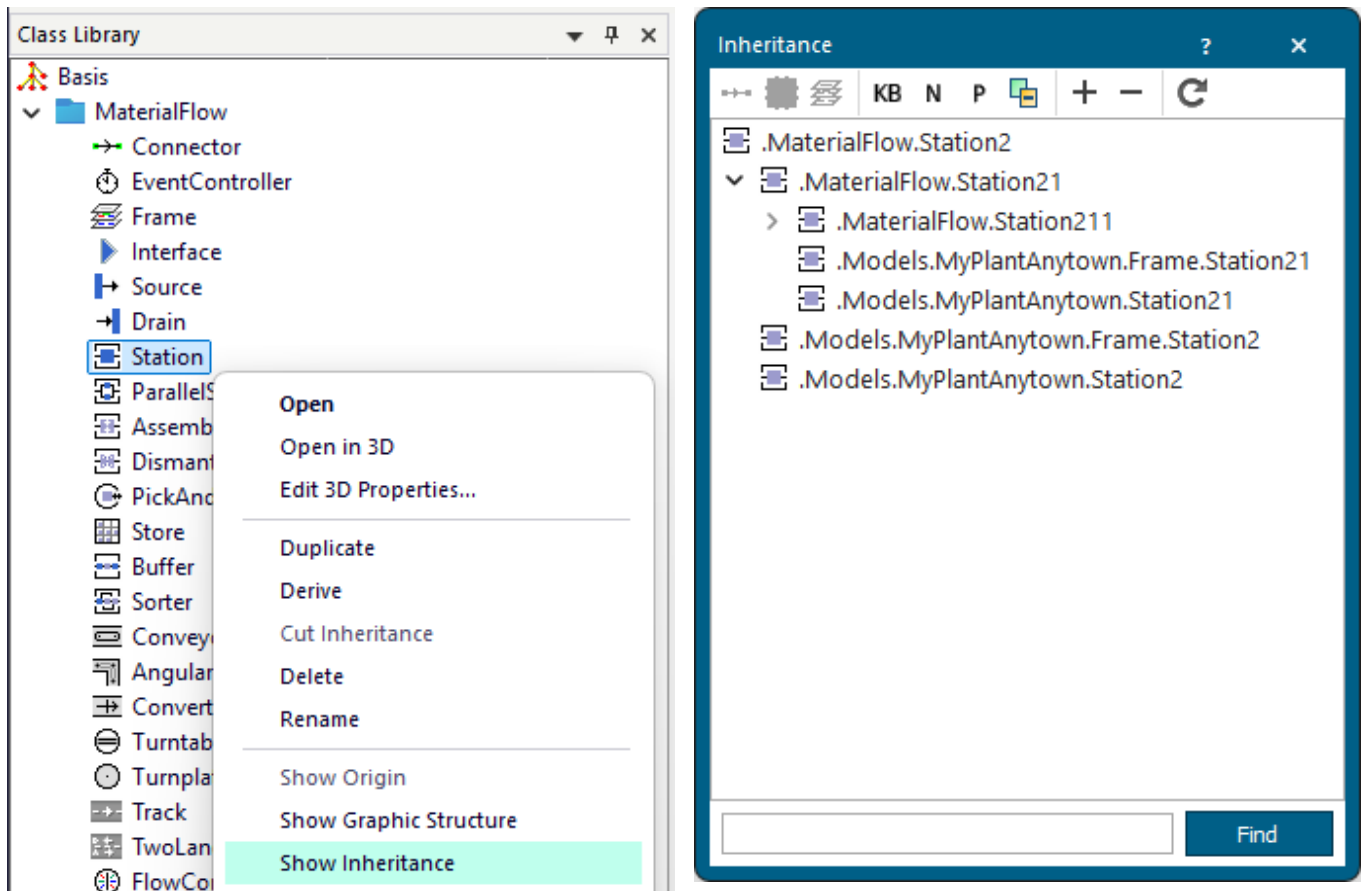
<https://youtu.be/DwZkGMxXJIA?si=FAz76pLhEAR2X9Tv&t=22>

Show Inheritance Relations in the Class Library

You can show inheritance relations in the *Class Library* in the dialog **Inheritance**.

The objects can use **class** and **origin relations** to inherit some, or all of their properties from other objects. To get an overview over which object inherits from which other object, right-click the object you are interested in in the *Class Library* and select **Show Inheritance**.

The dialog **Inheritance** show the paths of all objects that are derived from the selected object and thus inherit some or all of its properties from it.



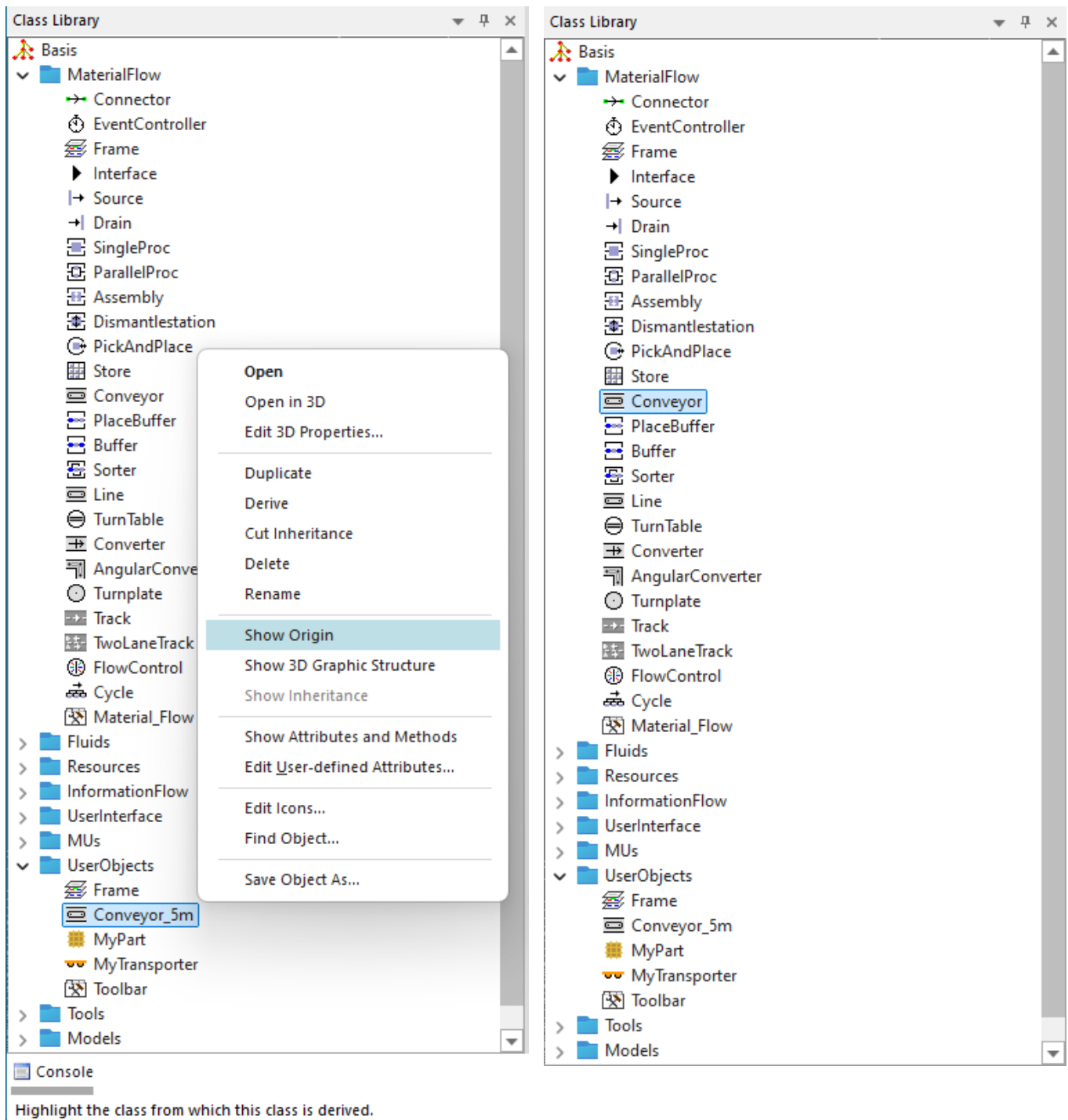
Back to [Using Inheritance](#)

Show the Origin of an Object in the Class Library

You can show the origin of an object in the *Class Library* in the dialog **Show Origin**.

The objects can use **class** and **origin relations** to inherit some, or all of their properties from other objects.

To select the object in the *Class Library* that the selected object was instantiated (derived) from, right-click the object you are interested in in the *Class Library* and select **Show Origin**.



You can repeat this to move up in the hierarchy until *Plant Simulation* does not select another object. Then you have reached the object from which the selected object was derived from originally, i.e., its origin.

Back to [Using Inheritance](#)

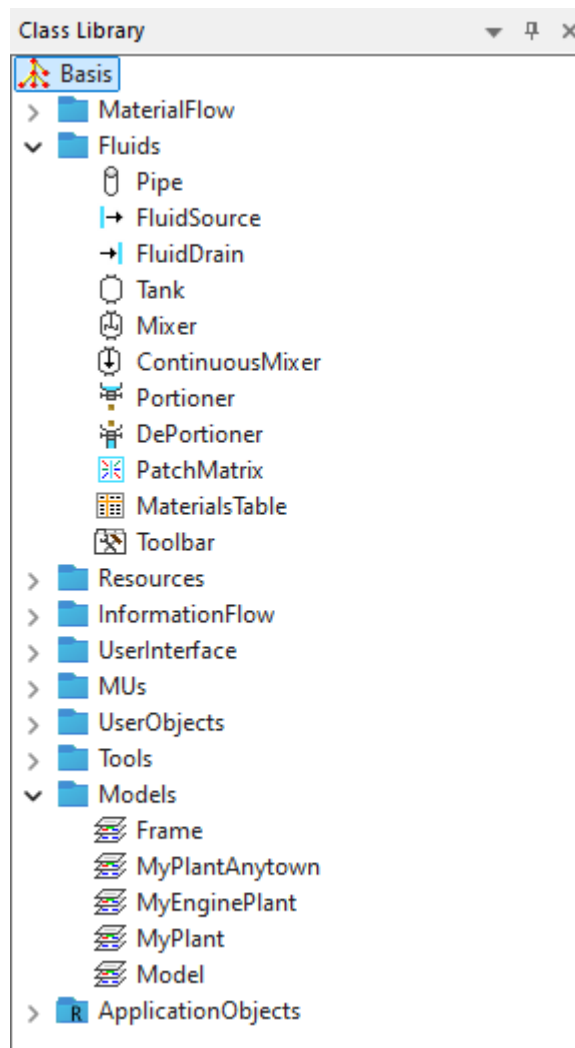
Working with Classes in the Class Library

Working with Classes in the Class Library

The *Class Library* shows the built-in objects in a hierarchical structure in folders and sub-folders.

By default the **Class Library** contains a folder with *MaterialFlow* objects, with *Fluid* objects, with *Resource* objects, with *InformationFlow* objects, with *MUs*, with *UserInterface* objects, with **UserObjects**, for *Models* and for *Tools*. When you start modeling, you will derive child objects from the built-in class objects or from class objects you, or a co-worker, built from scratch.

Plant Simulation initially opens the *Frame* named *Model* for creating your simulation model, and any other *Frames* you add later on, in the *Models* folder. The folders in the tree window share a number of properties with the other objects. Among others you can apply a number of *control methods*, for example the **Rename Control**, etc.



Note

If at all possible, we recommend to not change the standard settings of the built-in objects. If you customize them and then merge models later on or if you update the *Class Library* later on, your customized settings are going to be discarded and replaced by the standard settings. You then have to make your customizations again.

If you need objects with your own settings, we recommend to **derive** the respective object in the *Class Library* and then work with this class. *Plant Simulation* places duplicated and derived objects in the folder **UserObjects**.

When you add an add-in program, the *Class Library* also shows its objects—either loose or in a folder of its own—on the topmost level of the structure below **Basis**. **Basis** designates the *Class Library* itself, compare the anonymous identifier **basis**.

Compare [Work with Objects in the Toolbox](#)

Compare [Class \[general description\]](#), [Subclass \[general description\]](#), [Instance \[general description\]](#)

Go to [Classes, Subclasses, and Instances](#)

Go to [Using Inheritance](#)

Go to [Configure the Class Library](#)

Go to [Create a Folder Structure for Your Simulation Model](#)

Go to [Create and Work with Your Own Classes](#)

Back to [Introducing Classes](#)

Configure the Class Library

Configure the Class Library

You can configure the *Class Library* to best meet your needs. You can do this for new simulation models which you create and for any of your existing models.

This results in a lean **Class Library** and an uncluttered *Toolbox*, preserving precious screen space. You can:

- [Add Basic Objects to the Class Library or Remove Them from It](#). Naturally you can also add or delete objects at a later point in time.
- [Optimize](#) the model.

The option **Clean up class library** looks for classes that are not used in the model file and deletes them.

- [Add a Library or a Tool to or Remove it from the Class Library](#).

- [Add a Library, Which You Yourself Developed.](#)
- [Update a Library.](#)

When you select the command **File > New Model** for the first time, *Plant Simulation* opens the dialog **Manage Class Library** by default.

Compare [Using Inheritance](#)

Compare [Save a Folder or an Object and Load it into Another Model](#)

Compare [Save a Folder as a Library](#)

Compare [Update the Class Library](#)

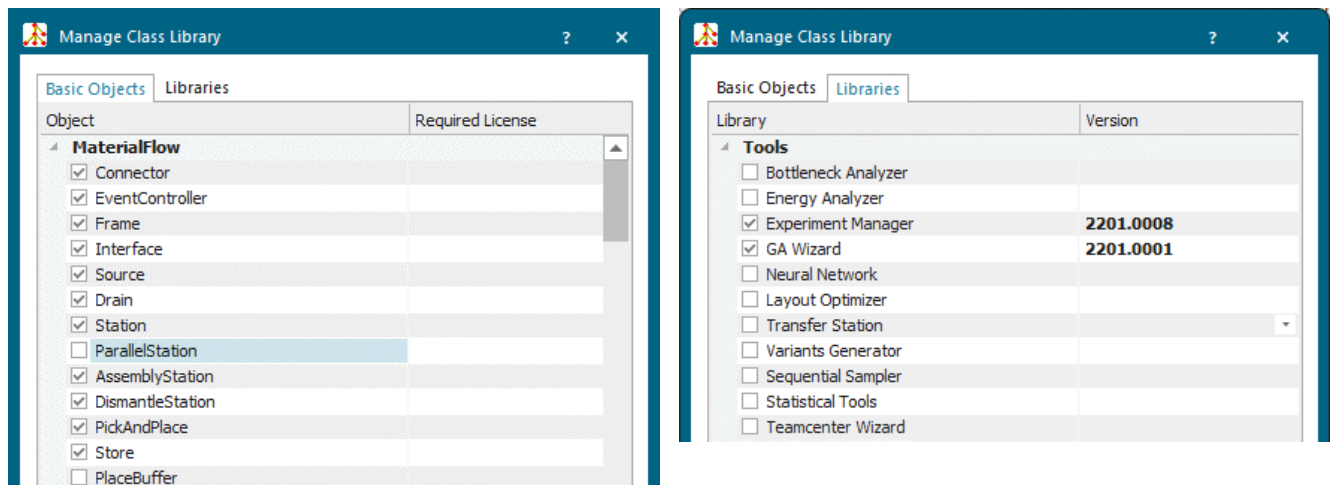
Back to [Working with Classes in the Class Library](#)

Add Basic Objects to the Class Library or Remove Them from It

You can add the **built-in objects**, which are part of your program package, to the *Class Library* and to the *Toolbox* in the dialog **Manage Class Library**.

Proceed as follows:

- Select the objects, which you need for creating the current and future simulation models.
Clear the check box for the objects, which you do not need.



- Click **Apply** to add the selected objects to the *Class Library* and to the *Toolbox* or to remove them from it.
- If you want *Plant Simulation* to always show this dialog, when you open a new model, select the check box **Always Show This Dialog When You Open a New Model**. Then you can configure each simulation

model to your liking. If you want to use the same settings for all new simulation models, clear the check box.

- If you want *Plant Simulation* to apply the settings you selected in the dialog to all new models, which you create hereafter, click **Apply to New Models**.
- Click **OK**, when you're done configuring the *Class Library*.

Back to [Configure the Class Library](#)

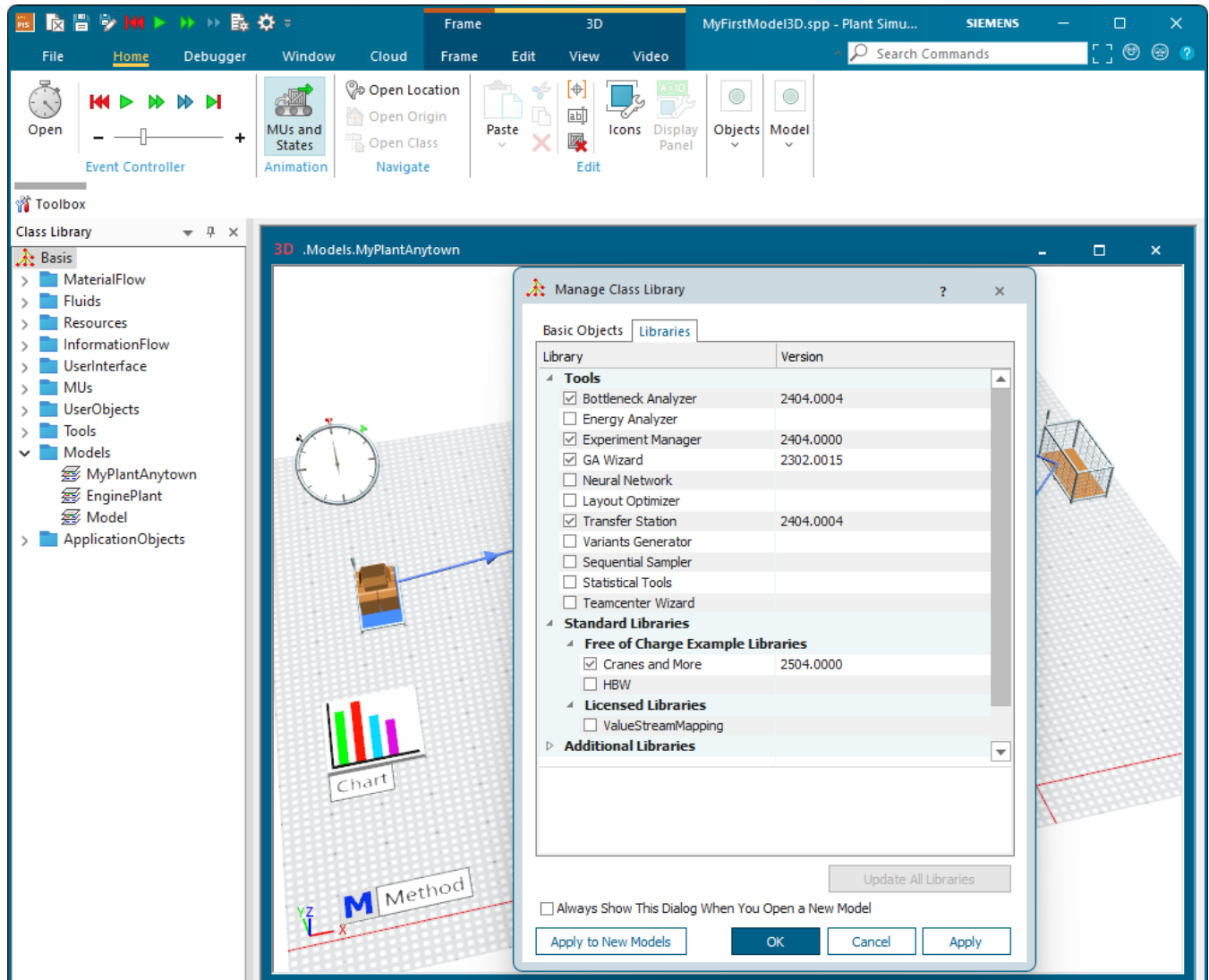
Add a Library or a Tool to or Remove it from the Class Library

You can add any of the **built-in tools or libraries**, which are part of your program package, to the *Class Library* and to the *Toolbox* or remove them from it from them.

Proceed as follows:

- Select one or several tools and libraries, which you need for creating the current and future simulation models.

Clear the check box for the tools or libraries, which you do not need.



- Click **Apply** to add the selected objects to the *Class Library* and to the *Toolbox* or to remove them from it.

Plant Simulation adds the selected folder to the *Class Library* and the *Toolbar* of the tool to the **Toolbox**.

Back to [Configure the Class Library](#)

Add a Library, Which You Yourself Developed

You can add one of the **libraries**, which you yourself developed, to the *Class Library* and to the *Toolbox*.

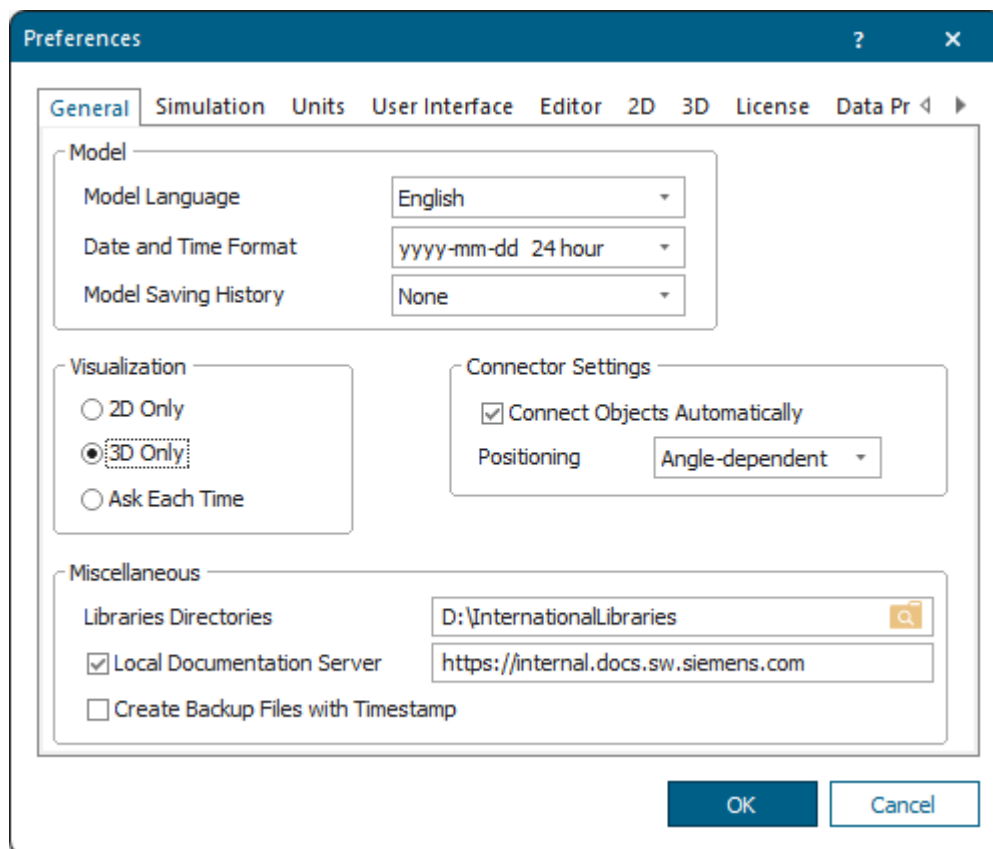
Proceed as follows:

- Tell *Plant Simulation* where to look for the library, which you are going to add. This location may be a folder on a local computer or on a network drive.

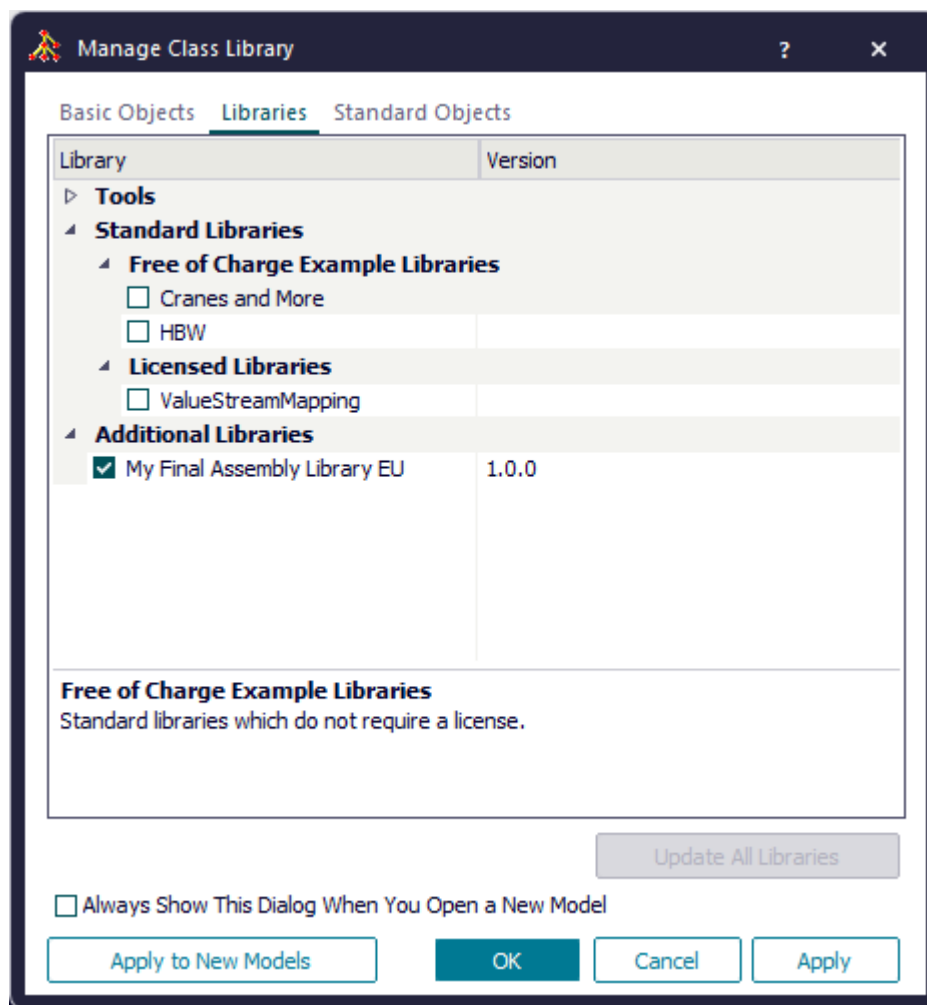
Select **File > Preferences** . Click  to the right of **Libraries Directories** and select the path.

You can also type in several directories separated by a semicolon. The *library manager* checks this folder/these folders for the most up-to-date version of a library and lets you download it to your hard drive from there. This enables you to add company-specific and project-specific libraries to your simulation model.

Click **OK**.



- The dialog **Manage Class Library** shows the library, **My Final Assembly Library EU** in our example, on the tab **Libraries** under **Additional Libraries**.



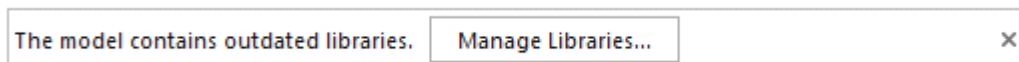
- Click **OK** to add the library to the *Class Library* and to the *Toolbox*.

Back to [Configure the Class Library](#)

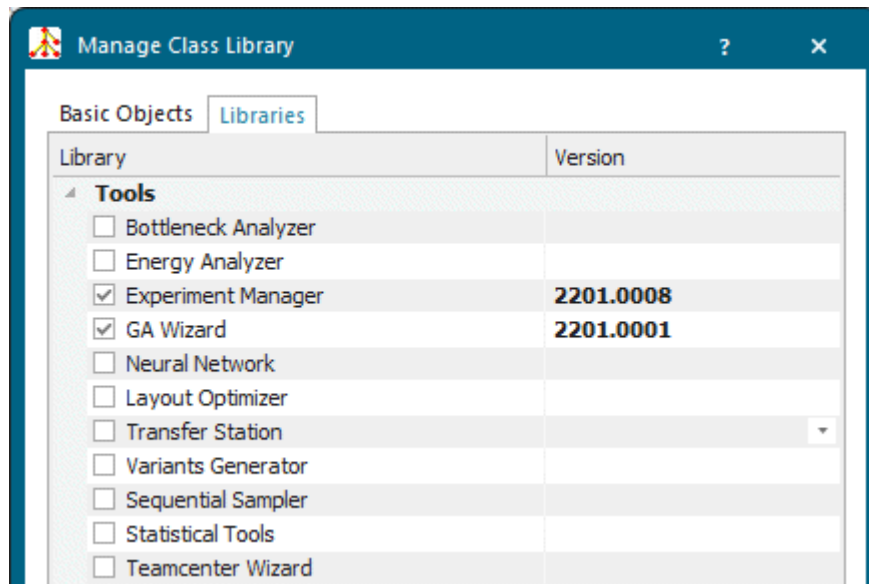
Update a Library

You can update an outdated library on your computer with a newer version with the *library manager*, i.e., the dialog **Manage Class Library**. Let's say you have to update a model, which you haven't worked with for a while and which contains a library.

If the *library manager* finds a newer version of the used library in the directory, which you entered under **File > Preferences > Libraries Directories**, the *Message Bar* show this message:

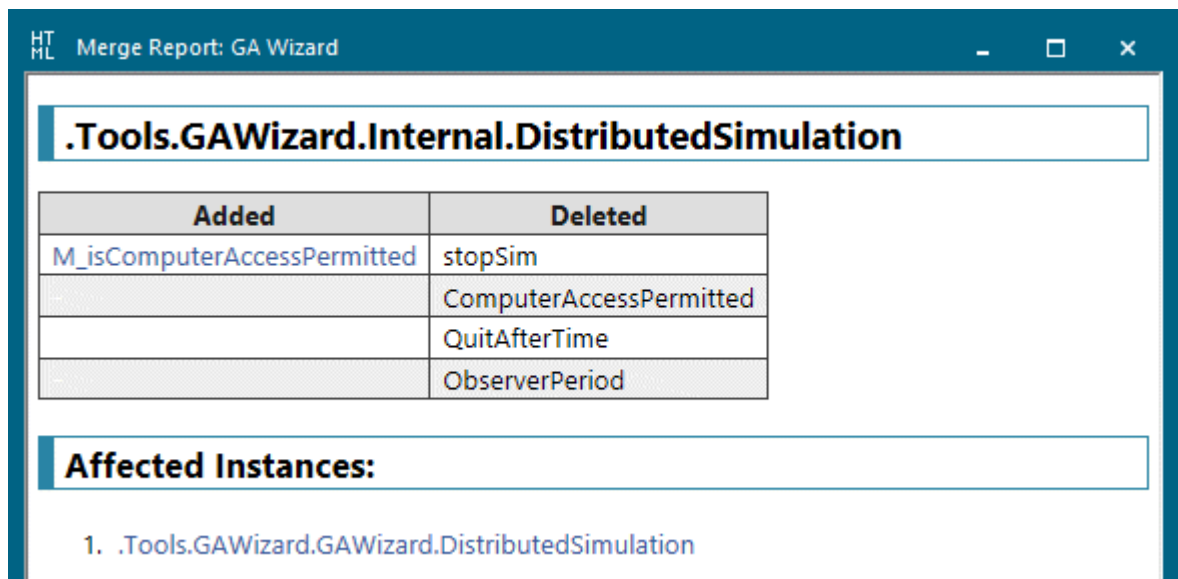


- Click **Manage Libraries**. The dialog shows libraries in red for which a newer version is available.



- In the dialog **Manage Class Library** click in the cell below version to show the version number of the newest version and select it if you want to update the *Class Library*. Click **OK**.

Plant Simulation show a **Merge Report** with detailed information about added and deleted objects, about the instances of the objects which were affected by the changes and which classes were deleted.



- Click **OK**, when you're done configuring the *Class Library*.

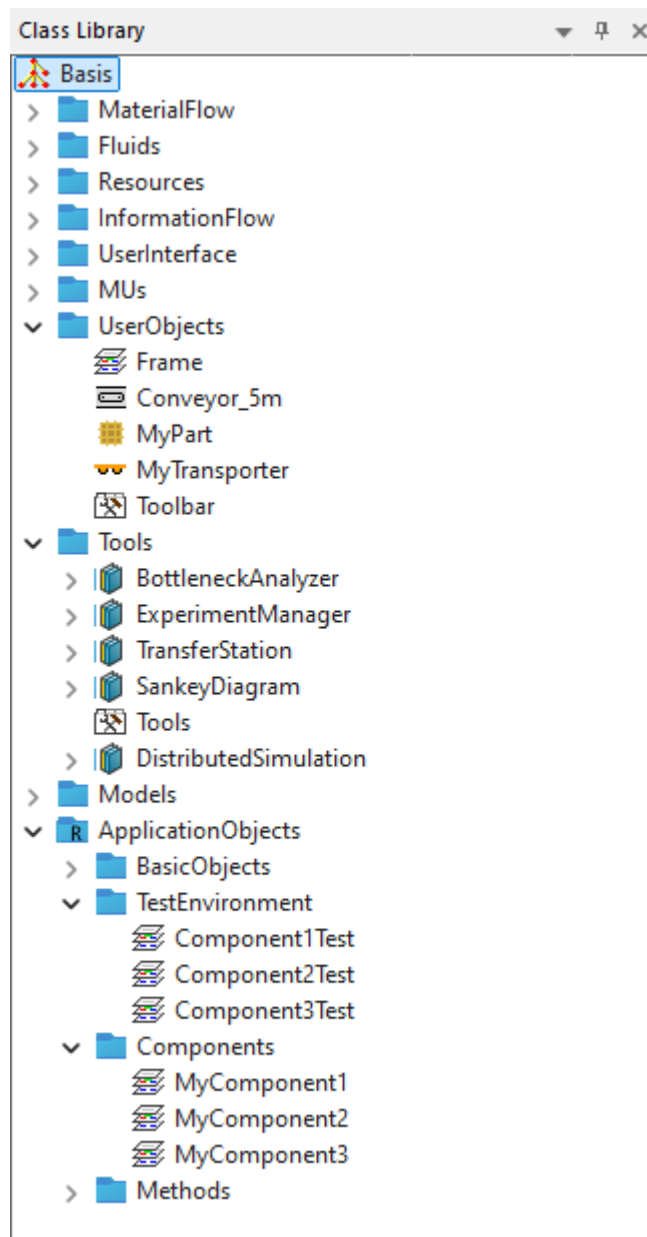
To update all libraries, which are not current, to the most current state, click **Update All Libraries**. *Plant Simulation* only shows the button when you opened a model. It does not show it when the dialog is opened while creating a new model. When you click the button, *Plant Simulation* selects the most recent available version number of all loaded libraries. It only actually updates the libraries when you click the **OK** or the **Apply** button.

Back to [Configure the Class Library](#)

Create a Folder Structure for Your Simulation Model

Before you start modeling, we recommend to think about the structure in the *Class Library* of your simulation model, as a clear structure will facilitate locating the objects.

To get an idea about structuring, you might want to take a look at the libraries that were installed, when you set up *Plant Simulation*. By default these are located in **C:\Program Files\Siemens\Tecnomatix Plant Simulation XX\Libraries**.



When you start modeling, we recommend to create new folders for the objects you use in your model, rather than store the objects within the built-in folders. Otherwise the risk of changing the built-in objects

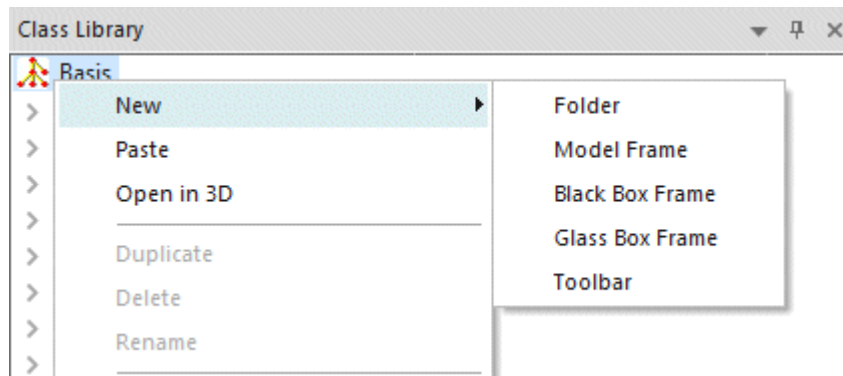
inadvertently might have confusing and unpredictable effects. This also facilitates saving your library as an object file, which you can then load into any existing simulation model.

We intended the folder **UserObjects** for this purpose, You can then add these objects to the tab **User Objects** in the **Toolbox**.

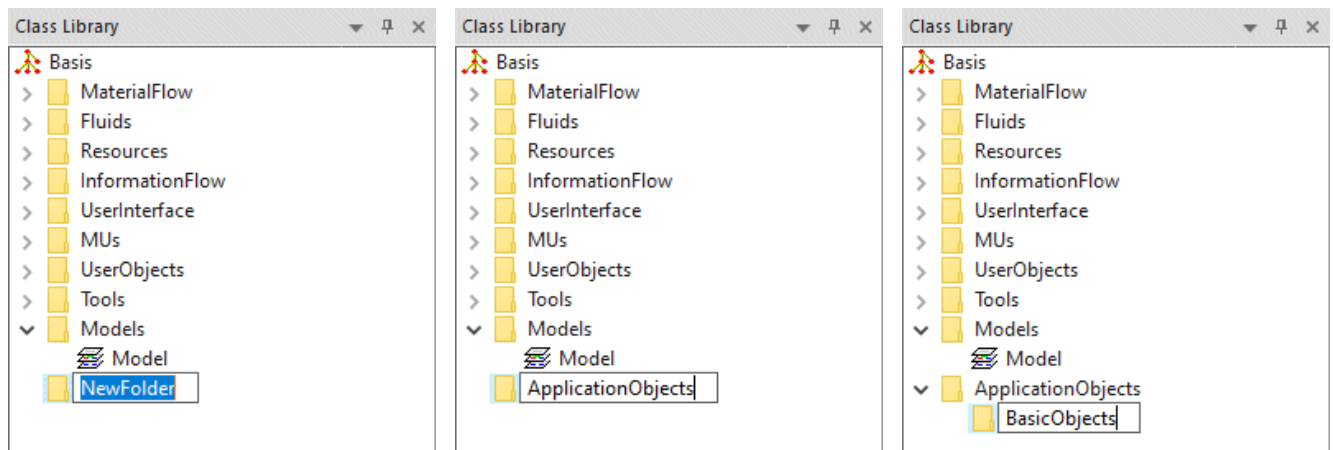
You can also create subfolders and thus structure your *Class Library* clearly. Proceed as described below.

If you do not want to use the folder **User Objects**, you can create your own folder:

- Right-click **Basis** in the *Class Library* if you want to insert the new folder on the same level as the built-in folders.
- Right-click any folder in the *Class Library* if you want to insert the new folder into that folder.
- Select **New > Folder**.



- **Rename** the folder: Click **NewFolder** once and type in a meaningful name. In our library we use **ApplicationObjects**.



We also recommend to create a subfolder for the basic, built-in objects that your simulation model uses, within the folder **ApplicationObjects**. In our library we named this subfolder **BasicObjects**. This makes your library independent of the language of the model that the user selects and of the naming

conventions of the built-in objects you use in your model. In addition, this simplifies loading the library into other models, as this mechanism makes merging objects superfluous.

Compare [Using Inheritance](#)

Compare [Save a Folder or an Object and Load it into Another Model](#)

Compare [Update the Class Library](#)

Back to [Working with Classes in the Class Library](#)

Set the Root Folder for Your Simulation Model

To prevent *Plant Simulation* from wasting RAM, you might, for example, not want to insert a *Method* that has numerous instances into numerous *Frames*. Instead, you can directly call this *Method*, in which you programmed an action that many objects use, from the *Class Library*. In this case, you can use the anonymous identifier *RootFolder* to avoid long paths.

When you use *RootFolder* in a *Method*, the *RootFolder* is a folder in the *Class Library* for which you have set the attribute **RootFolder** to `true`. Then, *Plant Simulation* searches for that folder, starting with the class of the *Method*, upward in the hierarchy of objects.

In your library, you will, as a rule, set the top folder that contains all your application objects as the rootfolder. This enables you to access any of the application objects you created, to access any of the dialogs you, or your colleagues designed, and so on.

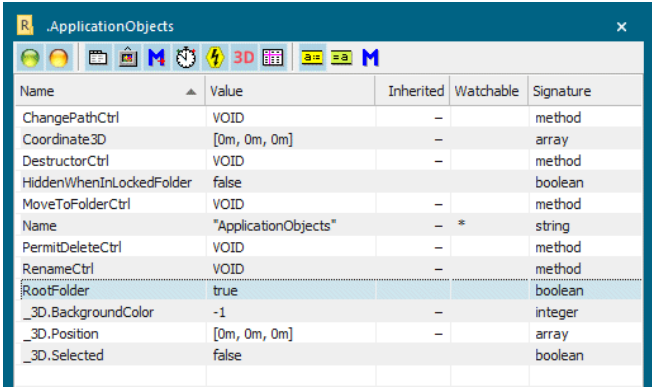
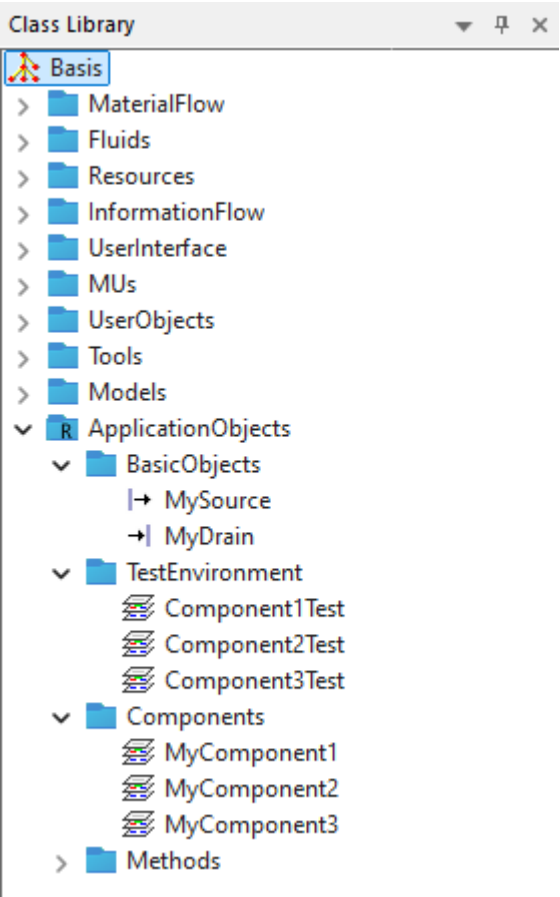
To set the root folder:

- Use the attribute *RootFolder*. You might, for example, enter this into a *Method*:

```
.ApplicationObjects.Transport.RootFolder := true
```

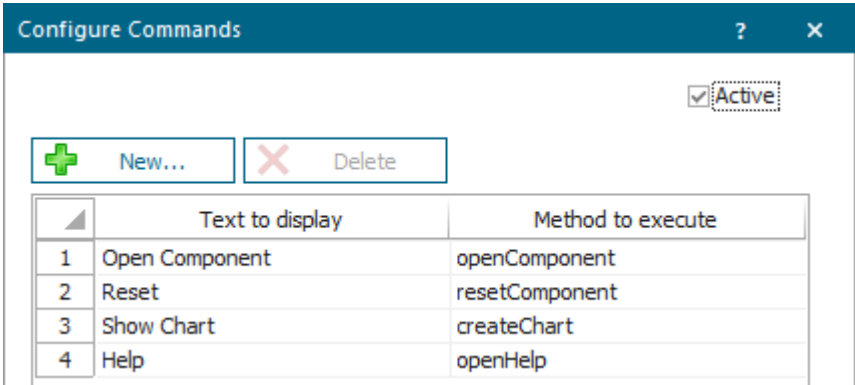
Or

- Right-click the folder you would like to use as the root folder in the *Class Library*. Select **Show Attributes and Methods**, and click in the window. Press **r** on the keyboard and scroll down to **RootFolder**. Double-click it, type `true` into the dialog that opens and click **OK**. *Plant Simulation* adds an upper-case **R** to the icon of the folder .



This example illustrates one of the usages of the root folder.

We designated the folder **ApplicationObjects** as the **RootFolder**. We inserted all objects that numerous other objects use into the *Frame* named **MyComponent1**. An example is the method *openObject* that opens a context menu for the objects and that provides the command **Open Component**. This enables us to access the method *openComponent* from this one central location by entering `RootFolder.MyComponent.openComponent` on the **Frame** ribbon tab **Context Menu > Context Menu** of the *Frames* for which we want to provide a context menu.



You can also use the anonymous identifier *RootFolder* within *Methods* in which you programmed controls. Here *Plant Simulation* looks for the folder for which you set the attribute *RootFolder*, starting with the class of the *Frame* into which you inserted the object in which you programmed the control.

Compare [Create a Folder Structure for Your Simulation Model](#)

Back to [Working with Classes in the Class Library](#)

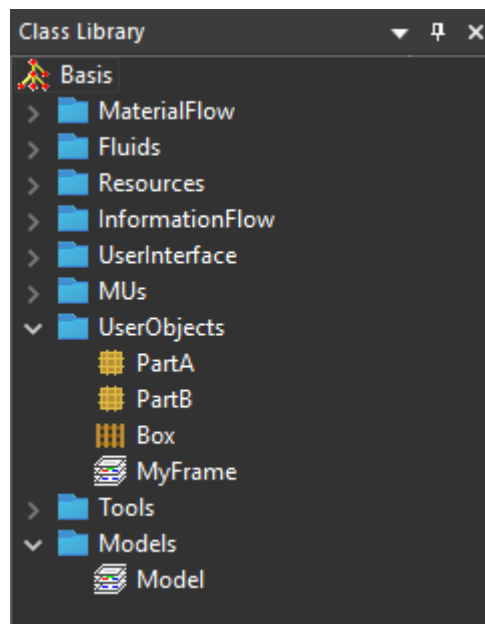
Creating User-defined Classes

Create and Work with Your Own Classes

If at all possible, we recommend to not change the standard settings of the built-in objects. If you customize them and then merge models later on or if you update the *Class Library* later on, your customized settings are going to be discarded and replaced by the standard settings. You then have to make your customizations again.

If you need objects with your own settings, we recommend to **duplicate** or **derive** the respective object in the *Class Library* and then work with this class. *Plant Simulation* places duplicated and derived objects in the folder [UserObjects](#).

The folder **UserObjects** provides these objects by default:



You can:

- [Create Your Own Classes](#)
- [Work with Your Own Classes](#)

Compare [Work with Objects in the Toolbox](#)

Back to [Working with Classes in the Class Library](#)

Create Your Own Classes

To create your own **customized classes**, you can either derive one of the built-in classes in the *Class Library* and modify its features, or you can build a class of your own from scratch, for example in a *Frame* in the *Class Library*.

To create a **class** that shares all of its features with the parent class:

- Do not change anything in the class object you want to derive an instance from. Then, right-click that object in its folder in the *Class Library* and select **Derive**.

This adds the derived class to the folder **UserObjects**. Then, rename the new class.

Or

- Select multiple objects in the *Frame* and drag-and-drop them onto the respective folder in the structure in the *Class Library* to create classes of these objects.

Or

- Hold down **Ctrl** and **Shift**, and click the object with the left mouse button. While holding the mouse button down, drag the object into another folder in the *Class Library*, and release both keys and the mouse button. Be aware that the class object keeps its previous name, when you proceed like this. Then, you can rename the new class.

To create a **subclass** that shares some, but not all, of its features with the parent class:

- Deactivate inheritance in the class object for the features you want to modify in each of the instances separately. Remember that a button that looks like this  designates that. The features with active inheritance will be passed on unchanged to the instances of the class object.
- Right-click the class in its folder in the *Class Library* that has the features you would like to use and select **Derive**. This adds the derived class to the folder **UserObjects**. Then, rename the new class.

Or

- Hold down **Ctrl** and **Shift**, and click the object with the left mouse button. While holding the mouse button down, drag the object into another folder in the *Class Library*, and release both keys and the mouse button. Be aware that here the class object keeps its previous name. Then, you can rename the new class.

To create a **class of its own**, which originally shares the settings of the original class, but does not keep an inheritance relation with it:

- Right-click the class in its folder in the *Class Library* that has the features you would like to use and select **Duplicate**.

This copies the selected object or model and creates a new class from it, severing all inheritance relations. *Plant Simulation* adds the duplicated class to the folder **UserObjects**. Changes you make to the original class object are not passed on to the copy you create. Or

- Hold down **Ctrl**, and click the object you want to duplicate with the left mouse button. While holding the mouse button down, drag the copy to another folder in the *Class Library*, and release **Ctrl** and the mouse button.

Compare [Classes, Subclasses, and Instances](#)

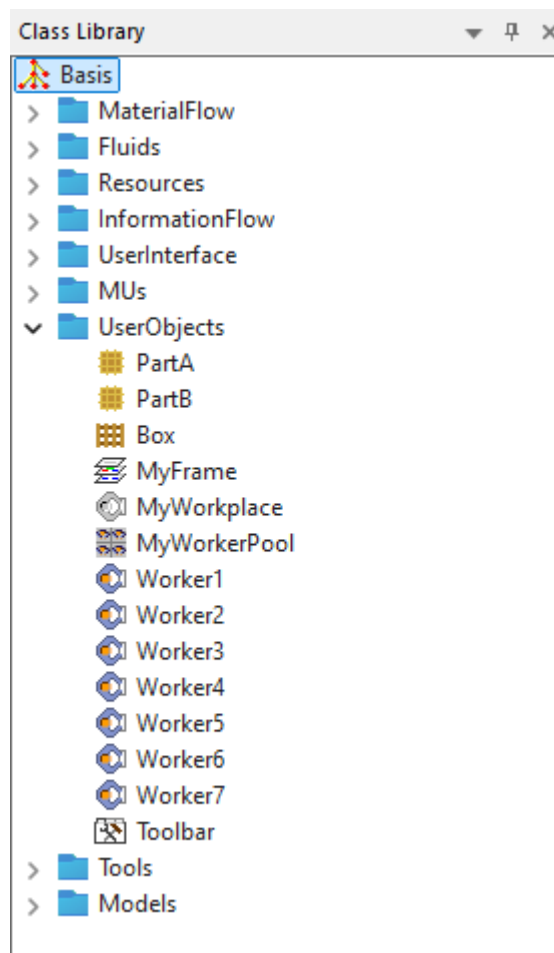
Compare [Using Inheritance](#)

Back to [Create and Work with Your Own Classes](#)

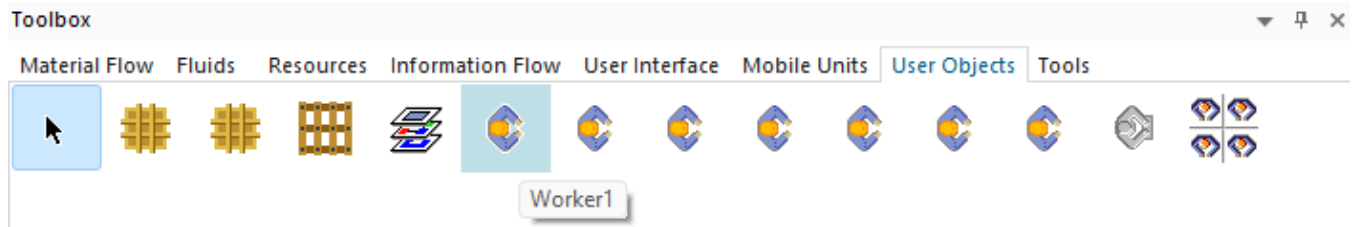
Work with Your Own Classes

After you created your own classes, you can work with them like you do with the built-in classes.

- *Plant Simulation* places duplicated and derived objects in the folder [UserObjects](#). Double-click the respective object to edit its properties. Insert the objects into your simulation model like you do with the built-in classes.



- You'll also want to add your classes to the toolbar **UserObjects**. Just drag the object class from the folder **UserObjects** to the toolbar **UserObjects**.



Back to [Create and Work with Your Own Classes](#)

Work with Folders, Frames, Objects in the Class Library

You can move, copy, and paste folders, *Frames*, and objects in the *Class Library*.

To **move a folder**  in the **tree window**:

- Select the folder you want to move in the tree window by clicking it with the left mouse button.
- Drag it to the folder above which you want to place it in the structure and drop it there.

To **move a folder**, a *Frame* or an object **into another folder** in the tree window:

- Click the folder, *Frame*, or object you want to move into another folder with the left mouse button to select it.
- Hold down **Shift**, drag the object to the destination folder and drop it there.

The move pointer looks like this .

To **copy a folder**, a *Frame*, or an object into another folder in the tree window:

- Click the folder, *Frame*, or object you want to copy into another folder with the left mouse button to select it.
- Hold down **Ctrl**, drag the object to the destination folder and drop it there.

The copy pointer looks like this .

To **merge folders**, hold down **Alt** and drag the replacing folder in the *Class Library* into the folder to be replaced and drop it there. *Plant Simulation* then forms the superset of both folders and merges those classes, which are present in both folders.

To **merge objects**, hold down **Alt** and drag the replacing object in the *Class Library* into the object to be replaced and drop it there.

To **arrange the order** in which *Plant Simulation* shows objects **within a folder in the tree window**, for example sort them by function or by name:

- Click the object with the left mouse button to select it.
- Hold the mouse button down, drag the object to the location of your choice in the folder and drop it there.

To **add icons** of objects to the toolbar in the tree window:

- Click the object with the left mouse button to select it.
- Drag the object to the object name of the toolbar  located at the bottom of the respective folder and drop it there.

To **rename an object**, do one of the following:

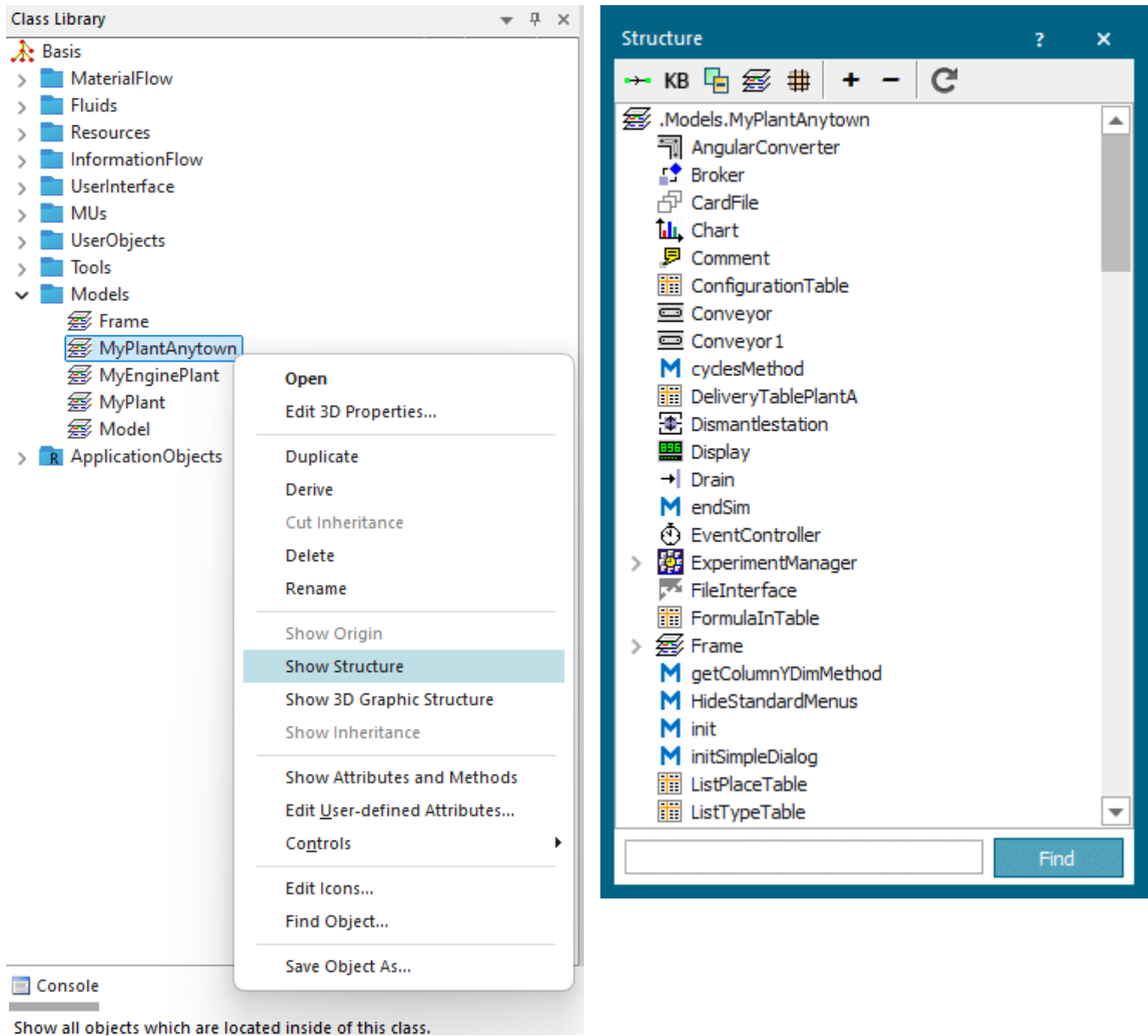
- Double-click it, and enter another name into the text box **Name**.
- Press **F2** and enter another name.
- Select **Rename** on the context menu and enter another name.
- To rename an open object, press **F4** and enter another name into the dialog **Rename**.

Back to [Working with Classes in the Class Library](#)

Show the Contents of a Frame in the Class Library

Plant Simulation does not show objects you insert into any *Frame* in the *Models* folder, as it only shows classes of objects in the *Class Library*, not instances of those classes.

To view the hierarchical structure of objects within a *Frame* , right-click the *Frame* in the *Class Library* and select **Show Structure** on the context menu.



Compare Find Objects and Text in Your Simulation Model

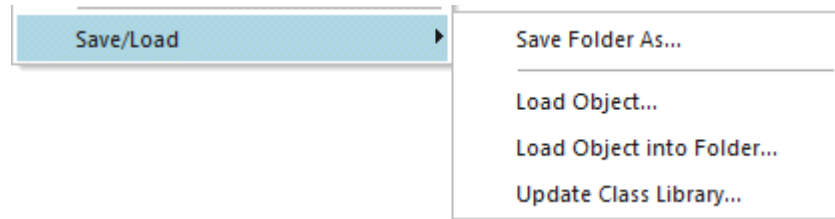
Back to Working with Classes in the Class Library

Saving a Folder or an Object and Loading it into Another Model

Save a Folder or an Object and Load it into Another Model

You can load a user-defined object, which may be a single object or an entire simulation model you created in a *Frame*, into other simulation models or share it with your co-workers.

As compared to an object file a **library** is a **collection of objects** for which you enter a **version**, which you **maintain over time** and which you share with your colleagues. The version number enables you to either update your simulation models with a newer version of the library or to stay with the present outdated version.



You can:

- [Save a Folder as a Library](#)
- [Save an Object or a Folder as an Object](#)
- [Load an Object or a Folder into Your Simulation Model](#)
- [Load an Object or a Folder into Another Folder](#)

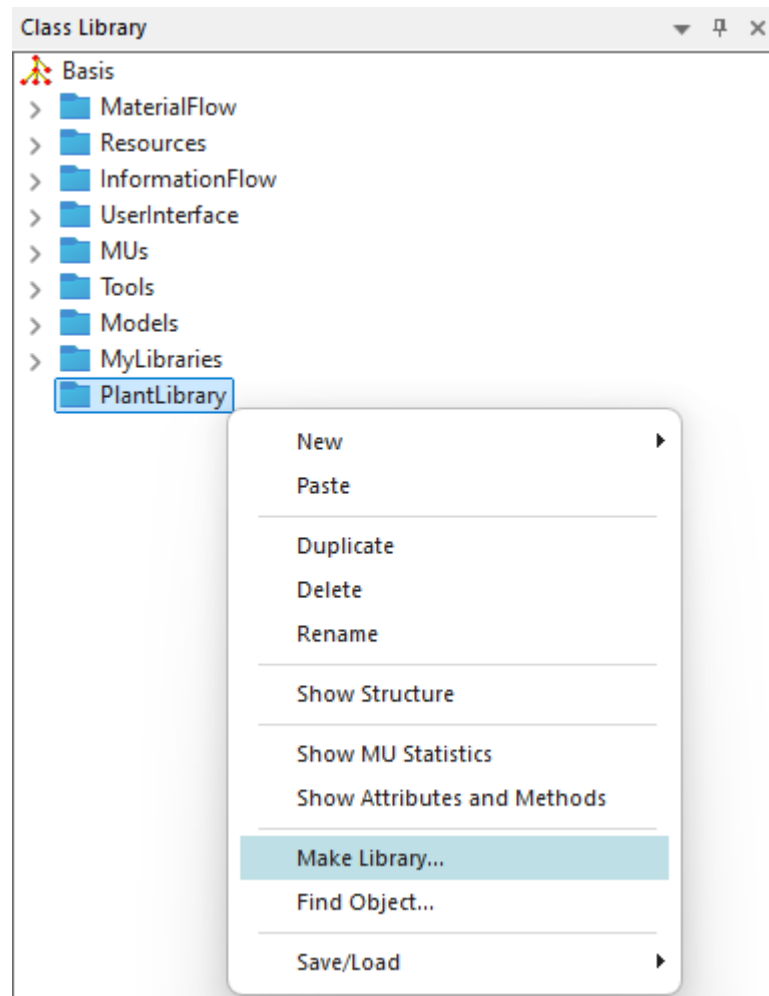
Compare [Configure the Class Library](#)

Compare [Update the Class Library](#)

Back to [Working with Classes in the Class Library](#)

Save a Folder as a Library

To save a folder as a *Plant Simulation* library (.pslib), right-click it in the *Class Library* and select [Make Library](#).



This opens the dialog **Library Information**.

- Type in the **Name** of the library. You can type in any combination of letters, numbers, spaces, and special characters.
- Type in the **Version** of the library. You can enter any number of number and letters separated by periods.
- Type in a meaningful **Description** of the library.
- Click **OK**.

The icon of a library in the *Class Library* looks like this  **PlantLibrary**.

To also use this library in other models, save the library as a file. To do so, click the library with right mouse button and select **Save/Load > Save Library As**. If you would like the library to be available in the dialog **Manage Class Library** later on, you have to select a location within the directory which you entered under **File > Preferences > General > Libraries Directories**.

You can then load this library into other simulation models and share it with your co-workers. To do so,

click **Manage Class Library**  on the **Home** ribbon tab in *Plant Simulation* window.

You can edit the information you typed in when you created the library by clicking the library with the right mouse button in the *Class Library* and by selecting **Edit Library Information**.

In addition, you can type in one or more **alternative paths** for your library. As a rule, you will not enter an alternative path.

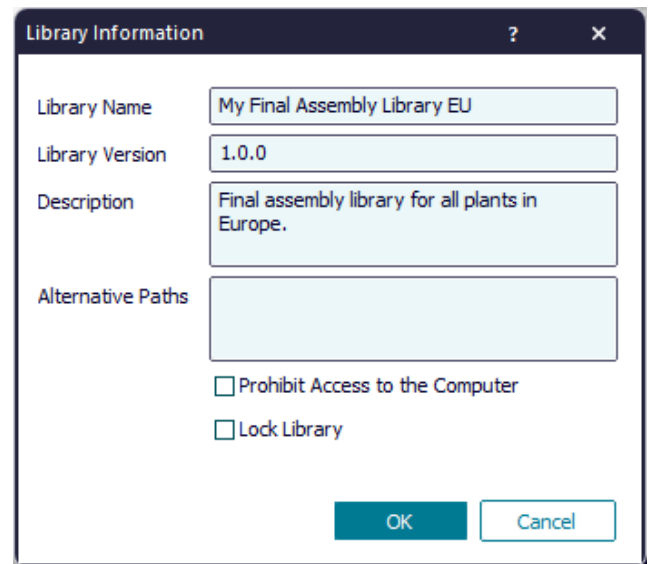
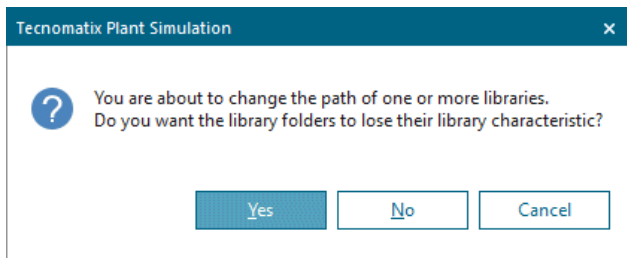
Plant Simulation identifies a library by its absolute path to the library folder in the *Class Library*. When you type in an alternative path, this absolute path will also be assigned to this library. This may be helpful for two reasons:

Procedure

1. When you rename the library folder in a new version of your library or when you move the library into another folder, the absolute path of the library changes.



Plant Simulation will from then on treat the library as another library. If you do not want this, you can type in the previous path to the library as an alternative path. This way you can update a model, for which the library is still located under the previous path, with the new library. For this model, the library remains at its previous location in the *Class Library*.



2. You can use alternative paths to assign several names in different languages to the library folder. We, for example, also assigned the German path `.Tools.EngpassAnalyse` language to the library **.Tools.BottleneckAnalyzer**. For this reason *Plant Simulation* considers both folders as the same library and can thus update both folders with the same library file.

Note

When you type in the name of a language between \$ signs in front of the alternative path, for example `$German$` or `$Japanese$`, *Plant Simulation* will automatically use this path, when you load this library into a model with this language, i.e., when the model did not already contain this library. The alternative path for the *BottleneckAnalyzer* is `$German$.Tools.EngpassAnalyse` for example.

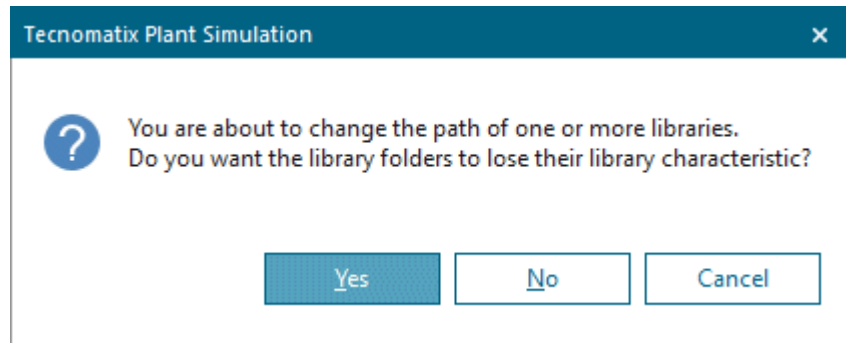
Note

You can type in any number of alternative paths. Separate alternative paths with a line break.

When renaming a library in the *Class Library*, pay heed to the messages that *Plant Simulation* displays!

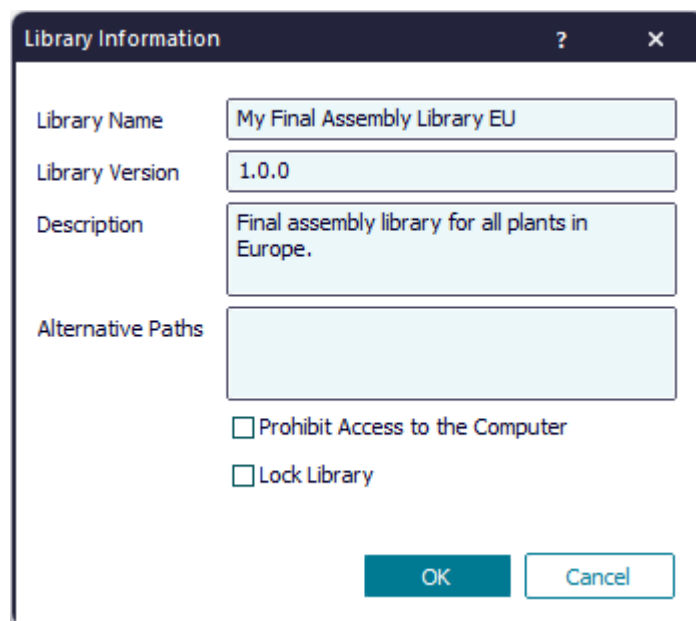


Click **No** in the dialog to retain the library properties of the folder. Clicking **Yes** will make the folder a normal folder again without any of the properties that designate a library.



If you want the renamed library to be treated as a different library from now on, click **Yes**.

Renaming the library folder does not change the name of the library as such, it will still have the same name as before.



Note

When you insert objects into a *Frame* of the library, you are only allowed to use objects from inside of your library or objects from other libraries. If your library uses objects from other libraries, *Plant Simulation* saves all dependent libraries into the .pslib file when you save your library.

Note

When you use a library in your simulation model or in a different library, we strongly advise against modifying the classes located inside the library. If, for any reason, you need to change the behavior of

a library class, you nevertheless should never modify the class itself, but derive it. *Plant Simulation* will place the derived class into a subfolder named **Public**. You may modify the derived class and use it in your simulation model.

When you later update the library – although the new version of the library does not have this **Public** folder – *Plant Simulation* will not delete the **Public** folder.

Compare [Configure the Class Library](#)

Compare [Update a Library](#)

Back to [Save a Folder or an Object and Load it into Another Model](#)

Save an Object or a Folder as an Object

Save a *Plant Simulation* object or a folder as a *Plant Simulation* object file.

To save an application object as an *Plant Simulation* object file (.psobj), right-click it in the *Class Library* and select [Save Object As](#).

Enter a name into the dialog **Save As** and specify a location. *Plant Simulation* saves the object, and all objects, which that object uses, together with their folders and paths.

To save a folder, which may contain an entire library, as an *Plant Simulation* object file (.psobj), right-click it and select [Save Folder As](#).

Enter a name into the dialog **Save As** and specify a location. *Plant Simulation* saves the folder and all objects it contains.

Back to [Save a Folder or an Object and Load it into Another Model](#)

Load an Object or a Folder into Your Simulation Model

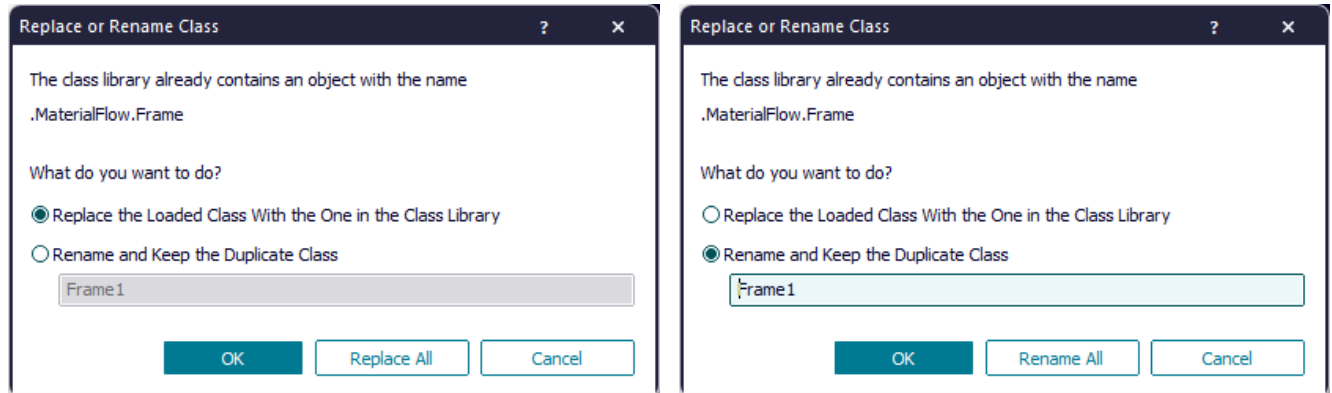
You can load an object or a *Frame/folder* into your open or into another simulation model.

To load an object or a *Frame/folder* into your open or into another simulation model, right-click any folder and select [Load Object](#). *Plant Simulation* loads the object with all of its existing paths and the folders, subfolder and classes the object inherit values from into the *Class Library* at the **Basis** level.

To preserve the existing objects in your simulation model, and to discard duplicate objects that are contained in the model you load, you will use **Load Object**, and then select to **Replace the Loaded Class with the One in the Class Library**.

To port a lower version of a simulation model or of a library to a higher version with modified and improved classes, you will use **Update Class Library**.

- Right-click any folder in the **Class Library** and select **Load Object** on the context menu.
- Select the object in the dialog **Open**.
- If an object or a *Frame* with the same name as the object to be loaded already exists in your simulation model, *Plant Simulation* opens the dialog **Replace or Rename Class**.



- To **Replace the Loaded Class with the One in the Class Library**, select the radio button. Then, *Plant Simulation* does not load the new object, but replaces it with the existing object and merges all inheritance hierarchies into one.
- To replace all duplicate objects, click **Replace All**.
- To **Rename and Keep the Duplicate Class**, select this.
- Enter a new name for the duplicate object into the text box.

Note

When the object to be replaced is a *Frame*, click **Replacement Mode** on the **Frame** ribbon tab of the *Frame*. Then, select if you want to **Merge** the contents of the new *Frame* with the existing one or if you want to **Exchange** the existing *Frame* with the new *Frame*.

Repeat this until all duplicate objects are renamed or replaced.

If you are unsure if both classes are identical, we recommend that you enter a new name and compare the classes. You can still replace the objects at a later point in time:

- Hold down **Alt**, click the replacing object and drag it over the object to be replaced while holding the left mouse button down.
- When replacing and merging is possible, a dialog opens where you can confirm that you want to replace the object or not replace it.

Instead of using **Load Object**, you can also drag an *.psobj* file from the *Windows Explorer* to the *Class Library*, and drop it there.

Back to [Save a Folder or an Object and Load it into Another Model](#)

Load an Object or a Folder into Another Folder

To load an object or a *Frame* that you saved as an object file (.psobj), into the folder in the *Class Library*, which you select, right-click the folder and select **Load Object into Folder**. *Plant Simulation* loads the object, with its complete structure of objects, which this object uses and which are used by it, into this new folder.

Note

As a rule, you will create a new folder before you load the object or the folder.

You will use **Load Object into Folder**, when you do not want to replace any of the existing objects and their hierarchical structure in your simulation model with the object/objects you are going to load.

The tools in the **Tools** folder in the *Class Library*, such as the *BottleneckAnalyzer*, use this technique and provide their own parametrized **BasicObjects** that are independent of the object classes in the built-in folders.

Back to [Save a Folder or an Object and Load it into Another Model](#)

Update the Class Library

Update a library, you and your coworkers are working on, with the most up-to-date classes you developed or to port a lower version of a simulation model or of a library to a higher version with modified and improved classes.

Proceed as follows:

- Save the object as an object file (.psobj): Right-click it and select **Save Object As**.
- Right-click any folder in the *Class Library* and select **Save/Load > Update Class Library**. This loads the object into the *Class Library* at the **Basis** level.

When loading the object file, *Plant Simulation* supposes that the classes in the file it is loading are the most recent ones, and replaces any existing classes with the same name in your model file. This replaces the old class with the new class, but attaches any instances of the old class to the new class, and thus preserves the settings you made in these instances. This applies for classes that are located in the hierarchy directly below the newly saved class.

For classes that were saved through an inheritance relation, and that are located outside of this hierarchy below a class, *Plant Simulation* opens a dialog, where you have to select to replace any duplicate classes or to rename them.

Compare [Save a Folder or an Object and Load it into Another Model](#)

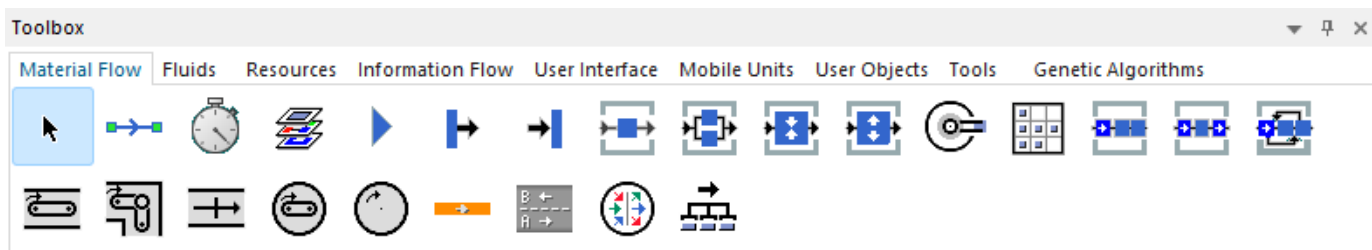
Back to [Working with Classes in the Class Library](#)

Working with Objects in the Toolbox

Work with Objects in the Toolbox

The *Toolbox* is a container for the different *Plant Simulation* toolbars that hold the objects of the *Class Library*.

By default, the **Toolbox** shows the built-in objects on the toolbars **Material Flow**, **Fluids**, **Resources**, **Information Flow**, **User Interface**, **Mobile Units**, and **Tools**. You find the toolbars for the objects at the very bottom of their folders next to the icon .



When you add an add-in program by clicking **Manage Class Library** on the **Home** ribbon tab, *Plant Simulation* also adds a tab/toolbar with the objects of this add-in.

- To add your own toolbars to the *Toolbox*, right-click the folder you want to add the toolbar to and select **New > Toolbar**.

Plant Simulation adds toolbars you define, toolbars with the objects of add-in products (**Home > Manage Class Library**), and toolbars of object libraries to the right of or below the tabs of the built-in objects in the *Toolbox*.

- To add an object to a toolbar you defined, drag the object from a folder of the tree view in the *Class Library* to the window/toolbar you defined into the *Toolbox* and drop it there.

Note

After you inserted an object from the *Toolbox* into a *Frame*, *Plant Simulation* automatically switches back to the tool **Select**  on the toolbars of the *Toolbox*.

- To change the name *Plant Simulation* show on the toolbar, go to the folder that contains the objects in the tree view, select the icon  at the very bottom of the folder, press **F2** and enter a term of your choice into the text box **Label**. If you do not enter a label, *Plant Simulation* shows **Toolbar**, i.e., the name of the toolbar in the *Toolbox*.
- To open the class of the object, right-click it and select **Show Class**.

You can:

- Add Objects to the Toolbox or Delete Them from It
- Copy Objects from Toolbar to Toolbar

- [Insert an Object from the Toolbox](#)

Compare [Toolbox](#)

Back to [Working with Classes in the Class Library](#)

Add Objects to the Toolbox or Delete Them from It

You can add a single or multiple objects to and remove them from a *Toolbar* in the *Toolbox* and change the order of the icons.

Proceed as follows:

- To add an object to a *Toolbar*, drag it from its folder in the *Class Library* to the toolbar of your choice and drop it there.

Note

When you derive an object from another object in the *Class Library*, modify the icon of the new object, and add it to the toolbar, you might have to resize the window before *Plant Simulation* displays the new icon.

- To add multiple objects to a *Toolbar*, select them and drag them from their folder in the *Class Library* to the toolbar of your choice and drop them there.
- To remove a button/object from the *Toolbar*, right-click it and select **Remove from Toolbar**.
- To change the order in which the icons of the objects are displayed on the *Toolbar*, click the button you would like to move and drag it to a different location on the toolbar.

Compare [New \[Folder, Frame, Toolbar\]](#)

Back to [Work with Objects in the Toolbox](#)

Copy Objects from Toolbar to Toolbar

To adapt simulation models that you created in previous versions of *Plant Simulation*, you can use the following technique to copy objects from one toolbar of the *Toolbox* to another.

This is especially handy for *Toolboxes* that contain a large number of objects:

- First, select the toolbar containing the object you would like to add to another tab.
- Right-click the folder whose tab you would like to add the object to in the tree window and select **Show Structure**. This opens the window **Structure**. Expand the folder here.
- Hold down **Ctrl** and click the object you would like to copy on its toolbar in the *Toolbox*. This selects the object in the tree window.
- Drag the object from the tree window to the icon of the toolbar  in the dialog **Structure**.